

SkyWx

실시간 항공기 · 위험기상 상황판

기획서 · 요구사항 분해 · 시스템 아키텍처 · DB ERD · API 명세 — 구현 착수 전 설계 패키지

문서 버전	v0.2 (v0.1 전면 개정, 2026-09-27)
대상 환경	macOS 26.6 로컬 개발 · Docker Compose · 개인 학습 · 포트폴리오 · 비상업
설계 우선순위	① 시스템 아키텍처의 안정성 ② 측정 가능한 성능 ③ 차별 보안 ④ 데이터 정직성(모르면 모른다)
권장 스택	nginx edge · Next.js 16 + MapLibre · Spring Boot 4.1 (Java 25) API/WS/엔진 · Python 3.13 수집기 · PostgreSQL 18 + PostGIS · Redis 8 (Streams)
v0.1과의 관계	데이터 소스 조사·약관 근거는 v0.1을 계승. 스택·구조·요구사항·데이터 모델·API는 이 문서가 대체(0.2절 변경 요약)

읽는 법. 외부 사실(한도·약관·버전·표준)에는 **[S번호]**를 붙였고, 부록 A에 출처 URL과 원문 인용문(출처당 1개, 2026-09-27 확인)을 실었습니다. 인용문을 찾지 못한 외부 사실은 넣지 않았습니다. 확인하지 못한 항목은 **“불확실”** 상자로, 설계 결정은 **“결정”** 상자와 ADR 번호로 표시했습니다. 용량·성능 수치 중 “추정”이라 적은 것은 구현 후 측정값으로 바뀌어야 합니다.

목차

0. 문서 개요와 v0.1 → v0.2 변경 요약

1. 프로젝트 기획 — 문제·목표·범위·차별점

2. 요구사항 분해 — 에픽 · 사용자 스토리 · 기능/비기능
요구사항 · 추적표

3. 외부 데이터 소스 설계

4. 시스템 아키텍처 — 컨텍스트 · 컨테이너 · 컴포넌트 · 데이터 흐름

5. 신뢰성 설계 — 장애 모델 · 폴백 · 백프레셔 · 재시작 안전성

6. 보안 설계 — 위협 모델 · 층별 통제 · 자동 검사

7. 성능 설계 — 예산 · 용량 산정 · 측정 방법

8. 데이터 모델 — ERD · 테이블 정의 · 보존 정책

9. API 명세 — REST v1 · WebSocket 프로토콜 · 오류 형식 · 운영 API

10. 핵심 알고리즘

11. 프론트엔드 설계

12. 기술 스택과 ADR 목록

13. 로컬 개발 환경 (macOS 26.6)

14. 테스트·품질 전략

15. 개발 로드맵 (WBS)

16. 리스크와 불확실 사항

부록 A 출처·인용문 · B 용어 · C 명령어

0. 문서 개요와 v0.1 → v0.2 변경 요약

v0.1은 “무료 API로 만들 수 있는가”를 확인하는 타당성 문서였습니다. v0.2는 **구현 착수 직전의 설계 패키지**로, 기존 포트폴리오(SmartCollab v2·AptLake·RoadRail·건설·부동산 위험 모니터·우체국 가는 길·Local Macro Dashboard)에서 반복해 온 설계 원칙 — 단일 공개 진입점, 조회/수집 프로세스 분리, 호출 예산 공유, 측정 기반 성능 개선, 위협 모델 기반 보안, 원천 불변·출처 추적, ADR — 을 실시간 스트리밍 도메인에 적용합니다.

항목	v0.1	v0.2 (이 문서)
백엔드	FastAPI 단일 프로세스(수집+API+WS)	Spring Boot 4.1 (Java 25) API·WS·공간 엔진 + Python 3.13 수집기 분리, Redis Streams로 연결 (ADR-001)
프론트	Vite SPA	Next.js 16 + BFF 프록시(브라우저는 API 주소·키를 모름) (ADR-002)
진입점	없음(개발 서버 직접)	nginx edge 단일 공개 포트, X-Forwarded-For 덮어쓰기, 요청 제한 (ADR-003)
영속화	Phase 3 선택	PostgreSQL 18 + PostGIS 를 MVP부터 사용: 항적 이력·재생·통계·감사 로그 (ADR-004)
기능	표시 + 교차 판정	+ 진입 예측(ETA) 알림 , 공항 비행 카테고리, 이력 재생, 통계, 운영 화면, 데이터 품질 게이트
요구사항	FR 12개 목록	에픽 7 → 스토리 21 → FR 34 · NFR 16(측정 방법 포함) · 추적표
데이터 모델	DDL 초안 3개	ERD + 테이블 14개 정의 + 보존·파티션·권한 정책
API	엔드포인트 8개 표	REST v1 24개 + WS 프로토콜 v1 + RFC 9457 오류 + 운영 API + OpenAPI 발췌
보안	비밀값 분리 위주	STRIDE 위협 모델 + 7개 층 통제 + CI 자동 검사

1. 프로젝트 기획

1.1 한 줄 정의

SkyWx는 전세계 항공기 위치(ADS-B), 기상 레이더, 항공 위험기상 경보(SIGMET), 공항 기상(METAR/TAF)을 한 지도에 겹치고, “지금 위험기상 구역 안에 있는 항공기”와 “N분 뒤 진입할 항공기”를 근거와 함께 실시간으로 찾아 주는 웹 서비스입니다.

1.2 문제 정의

문제

- 항공기 추적 서비스는 위치만, 기상 앱은 레이더만 보여 준다. “이 비행이 지금 뇌우 구역을 지나는가”는 사람이 두 화면을 겹쳐 보며 짐작해야 한다.
- SIGMET은 좌표 문자열과 고도대(FL)로 발표되어 비전문가는 읽기 어렵고, 유효시간이 지나면 조용히 사라진다.
- 무료 데이터는 공급자마다 형식·단위·한도·약관이 다르고, 2026년에만 세 곳이 조건을 바꿨다 (3.6절).

해결

- 서버가 여러 공급자를 **한도 안에서** 수집·정규화·검증하고, 브라우저에는 변경분만 밀어준다.
- SIGMET을 폴리곤 + 고도대 + 유효시간으로 구조화해 공간 인덱스로 항공기와 교차 판정하고, 진입/이탈/예상 진입을 **근거(어느 경보, 어느 고도, 언제)**와 함께 알린다.
- 공급자 교체 가능한 어댑터, 원천 보존, 품질 게이트, 출처 표기를 설계에 내장한다.

1.3 이 프로젝트로 보여 주려는 역량

역량	이 프로젝트에서 하는 일	확인 방법(이 문서의 위치)
실시간 시스템 설계	수집 → 스트림 → 인메모리 상태 → WebSocket 변경분 팬아웃, 백프레서·재접속·재시작 안전성	4장, 5장, 9.3절
공간 알고리즘	R-tree 교차 판정, 구면 dead reckoning, 진입 예측, PostGIS 이력 질의	10장, 8장
성능 공학	예산(p95·대역폭·메모리) 정의 → k6·JFR로 측정 → 병목 제거 기록	7장, 14장
보안 설계	STRIDE 위협 모델, edge 단일 진입, 최소 권한 DB 역할, 운영 화면 인증, CI 자동 검사	6장
데이터 정직성	원천 불변 보관, 품질 게이트, 출처·지연 표시, “추정”과 “관측” 구분	3.4절, 8장, 11장
폴리글랏 계약	Python 수집기와 Java API가 같은 스키마(JSON Schema)·같은 Redis 규칙을 공유, 계약 테스트	4.4절, 14장

1.4 범위

포함	제외(이번 버전)
- 관심 지역(기본 한반도 반경 250 NM) 10초, 전세계 120초 항공기 갱신 + 클라이언트 보간 - 레이더 과거 2시간 재생, SIGMET 폴리곤·고도대·유효시간 표시 - 교차 판정(현재) + 진입 예측(10분) 알림, 근거 패널 - 공항 METAR/TAF와 비행 카테고리(VFR/MVFR/IFR/LIFR) 색 표시 - 항적 이력 저장(72h 원해상도, 30일 1분 요약)과 시간 슬라이더 재생 - 통계 화면(FIR별 SIGMET, 시간대별 트랙픽, 알림 빈도) - 운영 화면(수집 상태·호출 예산·오류 로그·품질 게이트) — 로그인 필요	- 항공편 스케줄·지연·게이트(ADS-B로 얻을 수 없음, 무료 소스 없음) - 레이더 예측(nowcast) — 무료 소스 중단 [S16] - 일반 사용자 회원가입·개인화·푸시 알림(개인정보 수집 회피) - 모바일 앱, 다국어 - 상업적 이용·광고(약관 위반)

1.5 차별점 세 가지

- 겹쳐 보기가 아니라 판정** — 폴리곤·고도·시간 세 조건을 모두 검사한 결과를 “근거 카드”로 보여 준다(11.3절).
- 예측까지** — 현재 속도·방위로 10분 뒤 위치를 추정해 “진입 예상 · ETA 4분”을 먼저 알린다(10.3절). 추정값은 화면에 항상 “추정”으로 구분한다.
- 운영 가능한 상태로** — 호출 예산·지연·품질 게이트·오류를 운영 화면에서 바로 보고, 공급자 정책이 바뀌면 설정만으로 교체한다(5장, 9.5절).

1.6 성공 기준 (MVP 완료 정의)

≤ 10 s 관심 지역 갱신 주기(서버 수신→브라우저 반영 p95 ≤ 1.5 s)	≥ 30 fps 항공기 3,000대 표시 중 지도 조작(M 시리즈 Mac, Chrome)	≤ 50 % 모든 공급자 무료 한도 대비 하루 사용률	0 건 gitleaks·CodeQL·Trivy High 이상 미해결, 위조 헤더로 한도 우회
---	---	---	---

- `docker compose up` 한 번으로 6개 컨테이너가 뜨고 30분 안에 새 Mac에서 재현된다(13장).
- FR-01~FR-20(MUST)의 수용 기준을 자동 테스트로 통과한다(2.5 추적표).
- k6 부하 시험(WS 200 연결, REST 100 rps) 결과와 개선 전후 수치가 문서화된다(7장).
- 모든 화면 값에 출처와 수집 시각이 붙고, 추정값은 “추정” 표시가 있다.

2. 요구사항 분해

2.1 이해관계자와 사용자

역할	목표	대표 시나리오
일반 방문자(익명)	지도에서 항공기·기상·경보를 보고, 특정 항공기의 상황을 이해	인천 접근 중인 항공기가 SEV ICE 구역에 있는지 확인
항공·기상 학습자	과거 상황 재생, 통계로 패턴 파악	어제 18시 한반도 상공을 재생하며 SIGMET과 항적 비교
운영자(본인, 로그인)	수집 상태·예산·오류를 보고 공급자를 전환	adsb.lol 429 급증 시 폴링 주기 조정·폴백 확인
채용 담당자·리뷰어	설계 근거와 측정 결과를 문서·화면에서 확인	README → 설계서 → 운영 화면 → 테스트 리포트

2.2 에픽과 사용자 스토리

에픽	이름	사용자 스토리(US)
E1	실시간 항적	US-01 방문자로서 지도에서 현재 항공기 위치를 부드럽게 움직이는 아이콘으로 보고 싶다. US-02 항공기를 클릭해 호출부호·기종·고도·속도·수직속도·출처·수신 시각을 보고 싶다. US-03 좋아웃하면 전세계 항공기를 저주기로라도 보고 싶다.
E2	기상 레이어	US-04 레이더 지난 2시간을 타임라인으로 재생하고 싶다. US-05 SIGMET을 종류별 색과 고도대·유효시간으로 구분해 보고 싶다. US-06 공항을 클릭해 METAR/TAF와 비행 카테고리를 보고 싶다.
E3	위험기상 판정·알림	US-07 지금 SIGMET 안에 있는 항공기 목록을 근거와 함께 보고 싶다. US-08 곧 진입할 항공기를 ETA와 함께 미리 알고 싶다. US-09 알림이 경계에서 깜빡이지 않고, 진입/이탈이 한 번씩만 기록되면 좋겠다.
E4	이력·재생·통계	US-10 특정 시각의 상황(항적+SIGMET)을 재생하고 싶다. US-11 선택 항공기의 최근 항적 선을 보고 싶다. US-12 FIR별 SIGMET 빈도, 시간대별 트래픽, 알림 건수 통계를 보고 싶다.
E5	데이터 정확성	US-13 화면의 모든 값에 출처와 수집 시각이 있으면 좋겠다. US-14 데이터가 오래되면(지연) 화면이 그 사실을 알려 주면 좋겠다. US-15 추정값(보간·예측)은 관측값과 구분되면 좋겠다.
E6	운영·관측	US-16 운영자로서 공급자별 호출량·남은 예산·마지막 성공 시각을 보고 싶다. US-17 오류 로그를 원문 그대로 복사하고, 품질 게이트가 거른 데이터를 보고 싶다. US-18 설정(폴링 주기·공급자 순서)을 재시작 없이 바꾸고 싶다.
E7	플랫폼·보안	US-19 개발자로서 한 명령으로 전체를 띄우고 테스트하고 싶다. US-20 운영자로서 운영 화면은 로그인한 나만 볼 수 있어야 한다. US-21 누군가 API를 남용해도 외부 공급자 한도가 소진되지 않아야 한다.

2.3 기능 요구사항 (FR)

우선순위: **MUST** MVP 필수 · **SHOULD** MVP 포함 권장 · **COULD** 이후 단계. 수용 기준은 자동 테스트로 검증 가능한 형태로 적었습니다.

ID	US	우선	요구사항	수용 기준
FR-01	01	M	관심 지역 항공기 상태를 10 s 주기로 수집·정규화한다	연속 60분 동안 수집 간격 p95 ≤ 12 s, 위치 없는 레코드 0건 통과
FR-02	03	M	전세계 항공기 상태를 120 s 주기로 수집한다 (OpenSky 계정 크레딧 예산 내)	하루 크레딧 사용 ≤ 2,880(72%), 429 발생 시 다음 주기 자동 연장
FR-03	01	M	브라우저는 WebSocket으로 스냅샷 후 변경분(diff)만 받는다	연결 직후 snapshot 1건, 이후 10 s마다 diff, 30 s마다 전체 재동기화
FR-04	01,15	M	갱신 사이 클라이언트가 위치를 보관하고 60 s 초과 미수신은 반투명 처리한다	보관 함수 단위 테스트(정지·경도 ±180°·극지방 제외), stale 플래그 검증
FR-05	02	M	항공기 상세(호출부호·등록·기종·고도·속도·수직속도·squawk·출처·수신 시각)	REST 응답 필드 계약 테스트, 비상 squawk(7500/7600/7700) 강조
FR-06	04	M	레이더 프레임 목록(2h, 10분 간격)을 60 s 캐시로 제공, 타일은 브라우저 직접 로드	프레임 12개 이상, 타임라인 전환 ≤ 200 ms(사전 로드)
FR-07	05	M	국제 SIGMET을 5분 주기로 수집해 폴리곤·고도대·유효시간으로 구조화한다	AREA/AREAS/결측 링/미닫힌 링 fixture 6종 파싱 통과, 만료분 자동 제외
FR-08	07	M	항공기-SIGMET 교차 판정(수평 ∧ 고도대 ∧ 유효시간)	단위 테스트 12케이스(경계·base null·top null·지상 제외), 판정 시간 p95 ≤ 50 ms(10k대)
FR-09	09	M	알림 상태 머신(ENTERED/INSIDE/LEFT)과 히스테리시스(진입 2회, 이탈 3회)	경계 왕복 시퀀스 테스트에서 이벤트 2건만 발생
FR-10	07	M	알림 패널: 항공기·경보·고도·진입 시각·근거 표시	E2E: fixture 경보 안 항공기 3대가 패널에 1 s 내 표시
FR-11	13,14	M	모든 데이터에 provider-fetched_at·lag를 붙이고 화면 상단에 지연 배지	lag > 60 s(지역)/300 s(전세계)에서 배지 표시 E2E
FR-12	19	M	한 명령으로 전체 기동, fixture 모드로 외부 호출 없이 동작	SKYWX_FIXTURE_MODE=1 에서 E2E 전부 통과
FR-13	16	M	운영 화면: 공급자별 호출량·남은 예산·마지막 성공/실패·활성 공급자	REST /ops/providers 계약 테스트, 화면 E2E
FR-14	20	M	운영 화면 로그인(세션) 및 역할 분리	비로그인 /ops/* → 404, CSRF 없는 변경 요청 → 403
FR-15	21	M	공개 API·WS 요청 제한(IP·연결 단위)과 외부 공급자 호출 예산 분리	부하 도구로 IP당 121번째 요청 429 + Retry-After, 외부 호출량 불변
FR-16	01	M	공급자 어댑터(adsb.lol → adsb.fi → OpenSky) 자동 폴백과 복귀	1순위 3회 연속 실패 시 10 s 내 전환, 10분 후 복귀 시도(통합 테스트)
FR-17	11	M	항적 이력을 PostGIS에 저장(원해상도 72 h, 1분 요약 30일)	파티션 자동 생성·삭제 잡, 저장 지연 p95 ≤ 5 s
FR-18	11	M	선택 항공기 최근 항적(최대 2 h) 선 표시	REST /aircraft/{hex}/track p95 ≤ 100 ms(캐시 미스)
FR-19	14	M	공급자 전부 실패 시 마지막 스냅샷을 stale 표시로 유지	외부 차단 상태에서 화면이 비지 않고 배지 표시
FR-20	13	M	출처 표기(attribution) 상시 노출	E2E 스크린샷에 5개 출처 문구 존재
FR-21	08	S	진입 예측: 10분 dead reckoning 궤적이 SIGMET과 만나면 ETA와 함께 “예상” 알림	직선 진입 시나리오 ETA 오차 ≤ 60 s(합성 데이터), 예상 알림은 관측 알림과 다른 유형
FR-22	06	S	공항 클릭 시 METAR/TAF 원문·해석·비행 카테고리 색	카테고리 계산 단위 테스트(경계값 8건), 캐시 5/10분

ID	US	우선	요구사항	수용 기준
FR-23	10	S	이력 재생: 시각 t의 항적·SIGMET을 슬라이더로 재생($1\times\sim60\times$)	REST /replay p95 ≤ 300 ms(관심 지역 1시간 창)
FR-24	12	S	통계 화면 3종(FIR별 SIGMET, 시간대별 트래픽, 알람 건수)	일 1회 집계 잡, 조회 p95 ≤ 200 ms
FR-25	17	S	오류 로그 화면(원문 복사, 비밀값 자동 가림)	가림 규칙 단위 테스트(토큰·키 10패턴)
FR-26	17	S	품질 게이트: 비정상 레코드(속도 $> 1,200$ kt, 위치 점프 > 200 NM/10 s, 고도 $> 60,000$ ft) 격리·집계	격리 건수가 운영 화면에 표시, 격리 레코드는 화면에 미표시
FR-27	18	S	런타임 설정 변경(폴링 주기·공급자 순서·지역)을 운영 API로 반영	변경 후 다음 주기부터 적용, 감사 로그 기록
FR-28	05	S	미국 SIGMET/G-AIRMET 병합(전세계 뷰 확장)	중복 제거 키 테스트
FR-29	02	S	항공기 정적 정보 캐시(등록·기종)를 첫 관측 시 저장	동일 hex 재관측 시 DB 조회 없이 캐시 적중
FR-30	01	C	사용자 지정 관심 영역(브라우저 로컬 저장, 서버 무저장)	—
FR-31	04	C	한국 고해상도 레이더(기상청 API허브) 레이어	포맷 조사 후 별도 설계
FR-32	02	C	선택 항공기 위치의 Open-Meteo 현재 기상	클릭당 1회, 0.1° 반올림 캐시
FR-33	12	C	통계 CSV 내려받기	—
FR-34	04	C	NASA GIBS 구름 위성 레이어	—

2.4 비기능 요구사항 (NFR) — 목표와 측정 방법

ID	범주	목표	측정 방법
NFR-01	지연	수집기 수신 $\rightarrow$ 브라우저 반영 p95 ≤ 1.5 s (관심 지역)	메시지에 수집 시각 포함, 브라우저에서 차이 측정 (Playwright 100회)
NFR-02	REST 성능	스냅샷·SIGMET·상세 API p95 ≤ 80 ms(캐시 적중), ≤ 300 ms(미스), 100 rps	k6, Docker 로컬, 3회 반복 중앙값
NFR-03	WS 팬아웃	동시 200 연결에서 diff 전송 지연 p95 ≤ 500 ms, 메모리 ≤ 512 MB(api)	k6 websockets [573], JFR· docker stats
NFR-04	프론트	3,000대 표시 시 지도 조작 ≥ 30 fps, 첫 화면 JS ≤ 400 KB(gzip), LCP ≤ 2.5 s	Chrome DevTools 성능 패널·Lighthouse(캐시 없음 5회 중앙값)
NFR-05	엔진	10,000대 $\times$ SIGMET 200개 교차 판정 ≤ 50 ms	JMH 마이크로벤치
NFR-06	외부 한도	공급자별 무료 한도 대비 $\leq 50\%$ (OpenSky는 계정 크레딧 72% 예외 명시)	운영 화면 일일 집계, 7일 관측
NFR-07	가용성	공급자 1곳 장애 시 서비스 지속, 전부 장애 시 stale 표시로 지속	통합 테스트(WireMock/respx로 5xx·429 주입)
NFR-08	재시작 안전	api-collector 강제 종료 후 60 s 내 정상 복귀, 스트림 메시지 유실 0(at-least-once)	kill -9 후 Redis PEL 재처리 확인 테스트
NFR-09	데이터 보존	원해상도 72 h, 1분 요약 30일, 파티션 자동 관리, DB 크기 ≤ 10 GB(로컬)	일일 잡 로그, pg_total_relation_size
NFR-10	보안	gitleaks·CodeQL·Trivy·npm/pip audit High 0건, 위조 X-Forwarded-For 3경로 차단	CI 게이트, E2E
NFR-11	접근 통제	운영 API는 세션+CSRF+역할, 비인가 요청은 404(존재 여부 비공개)	통합 테스트

ID	범주	목표	측정 방법
NFR-12	관측성	공급자·스트림·WS·DB 지표를 Prometheus 형식으로 노출, 요청 추적 ID	/actuator/prometheus 스크레이프 확인
NFR-13	재현성	새 Mac에서 30분 내 기동, 환경 차이는 .env로만	체크리스트 실측
NFR-14	테스트	백엔드 라인 커버리지 $\geq 80\%$, 계약 테스트로 Python $\leftrightarrow$ Java 스키마 동기화	JaCoCo·pytest-cov, CI
NFR-15	준수	출처 표기 5종 상시, 비상업, 공급자 UA 설정	E2E 스크린샷, 설정 검사 테스트
NFR-16	정직성	추정값(보간·예측)은 항상 관측값과 시각적으로 구분, 결측은 “—”	컴포넌트 테스트

2.5 추적표 (FR → 컴포넌트 → 테스트)

FR	collector	api-ingest	api-engine	api-ws	api-rest	web	db	검증 테스트
FR-01/02/16	●						●	pytest 어댑터·백오프·폴백, 계약 fixture
FR-03/04		●		●		●		JUnit WS 통합, Vitest 보간
FR-05/18/29		●			●	●	●	REST 계약, Testcontainers
FR-06	●				●	●		pytest, Playwright 타임라인
FR-07/28	●	●					●	pytest 파서 fixture 6종
FR-08/09/10/21			●	●	●	●	●	JUnit 엔진 12케이스, JMH, E2E 패널
FR-11/19/20	●	●		●		●		E2E 배지·attribution
FR-12	●	●						fixture 모드 E2E 전체
FR-13/14/15/25/27	●			●	●	●	●	Security 통합, k6 한도
FR-17/23/24		●			●	●	●	Testcontainers PostGIS, 파티션 잡
FR-22	●				●	●	●	카테고리 경계값 8건
FR-26	●				●	●	●	게이트 규칙 단위 테스트

3. 외부 데이터 소스 설계

3.1 선정 요약 (v0.1 계승, 근거 유지)

역할	서비스	키	무료 한도	이용 조건	근거
항공기 (1순위)	adsb.lol	불필요	현재 레이트리미트 없음(향후 피더 API 키 예정)	ODbL 1.0, 모두에게 공개	[S6][S7]
항공기 (2순위)	adsb.fi	불필요	초당 1회	개인·비상업, 출처 표기	[S9]
항공기 (전세계)	OpenSky Network	OAuth2	계정 4,000 크레딧/일, 전세계 1회 4크레딧	연구·비상업, 운영 목적 REST는 서면 동의	[S1][S2][S3]
SIGMET·METAR·TAF·공항	AviationWeather.gov	불필요	분당 100회, 400건 상한, CORS 불가	미국 정부 공개 데이터	[S12][S13]
레이더 타일	RainViewer	불필요	IP당 분당 100회, 줌 7, 과거 2 h	개인·교육 전용, 링크 출처	[S16][S17][S18]
배경지도	OpenFreeMap + MapLibre	불필요	무제한	출처 표기	[S22][S23][S24]
지점 기상(선택)	Open-Meteo	불필요	일 10,000 미만	비상업, CC-BY 4.0	[S14][S15]
한국 레이더 (후순위)	기상청 API허브	인증키	합성 5분·500 m	본인 사용	[S19][S20]

3.2 공급자 어댑터 계약

수집기의 모든 어댑터는 같은 인터페이스를 구현하고, 결과는 `schemas/aircraft_state.v1.json` (JSON Schema)에 맞아야 스트림에 실립니다. 같은 스키마 파일을 Java 쪽 계약 테스트가 읽어 대조합니다(14.3절).

```
class AircraftProvider(Protocol):
    name: str # "adsb_lol" | "adsb_fi" | "opensky"
    supports_global: bool
    async def fetch_region(self, lat: float, lon: float, radius_nm: int) -> ProviderResult: ...
    async def fetch_global(self) -> ProviderResult: ...

@dataclass
class ProviderResult:
    states: list[AircraftState] # 정규화 완료(단위: ft, kt, ft/min)
    fetched_at: datetime # 서버 수신 시각(UTC)
    raw_ref: str # 원천 보관 경로(raw/.../{ts}.json.gz)
    budget: BudgetInfo | None # X-Rate-Limit-Remaining 등
```

공급자	엔드포인트	정규화 규칙
adsb.lol	GET <code>https://api.adsb.lol/v2/point/{lat}/{lon}/{radius}</code> (reads v2)	<code>alt_baro=="ground"</code> → <code>on_ground</code> , <code>alt_ft=0</code> ; <code>seen_pos</code> 로 관측 시각 역산; 단위 그대로(ft, kt, ft/min) [S8]
adsb.fi	GET <code>https://opendata.adsb.fi/api/v3/lat/{lat}/lon/{lon}/dist/{nm}</code>	reads v2 호환, 초당 1회 토큰 버킷 [S9]
OpenSky	GET <code>/api/states/all</code> (전세계) · OAuth2 client credentials, 토큰 30분	<code>m</code> → <code>ft</code> ×3.28084, <code>m/s</code> → <code>kt</code> ×1.94384, <code>m/s</code> → <code>ft/min</code> ×196.85 [S1][S38]

공급자	엔드포인트	정규화 규칙
AWC	/api/data/isigmet, /airsigmet, /metar, /taf, /airport	커스텀 UA, 분당 ≤ 20회로 자체 제한, AREAS 결측 링 제거 [S12][S13]
RainViewer	GET https://api.rainviewer.com/public/weather-maps.json	host-past[] 그대로 전달, 60 s 캐시 [S17]

3.3 호출 예산표(하루)

공급자	계획 호출량	한도	사용률	초과 시 동작
adsb.lol	8,640회(10 s)	명시 없음 [S7]	—	429 → 지수 백오프(2→300 s), 3회 연속 실패 → adsb.fi
adsb.fi(폴백)	≤ 8,640회	초당 1회 [S9]	≤ 10%	토큰 버킷으로 초당 1회 상한 고정
OpenSky	720회 × 4 = 2,880 크레딧	4,000 [S1]	72%	남은 크레딧 < 400이면 전세계 수집 중단, 다음 날 재개
AWC	SIGMET 288 + METAR/TAF ≤ 2,000	분당 100 [S12]	< 2%	분당 20회 자체 상한
RainViewer JSON	1,440회	분당 100/IP [S16]	1%	캐시 유지, 실패 시 마지막 프레임 목록 유지

결정 — 예산은 Redis 한 곳에서 (ADR-005)

공급자별 하루 예산을 Redis 키(budget:{provider}:{yyyymmdd})와 Lua 스크립트 “예약→사용→미사용분 반환”으로 관리합니다. 수집기가 여러 작업으로 동시에 돌아도, 운영자가 수동 호출을 해도 한도를 넘지 않습니다. 남은 예산은 운영 화면과 Prometheus 지표로 노출합니다.

3.4 원천 보관과 품질 게이트

- **원천 불변 보관:** 모든 응답 본문을 raw/{provider}/{yyyymmdd}/{hhmmss}.json.gz 로 72시간 보관(로컬 볼륨). 파서를 고치면 원천으로 재처리할 수 있다.
- **게이트 규칙(격리, 화면 미표시):** 위치 없음 · 속도 > 1,200 kt · 고도 > 60,000 ft · 10 s 내 위치 점프 > 200 NM · 관측 시각이 미래(> +30 s) · SIGMET 폴리곤 점 < 3.
- **격리 집계:** 규칙별 건수를 quality_event 에 기록해 운영 화면에서 본다. 게이트가 걸러 낸 데이터는 삭제하지 않고 원천에 남는다.
- **출처 추적:** 모든 스트림 메시지와 DB 행에 provider, fetched_at, raw_ref 를 붙인다.

3.5 선택하지 않은 소스

Flightradar24(무료 접근 없음 [S53]), FlightAware AeroAPI(월 \$5 한도 [S54]), airplanes.live(비상업도 연락 필요 · 2026-08 이후 403 보고 [S10][S11]), 카카오맵(한국 한정 [S26]), OSM 공식 타일(SLA 없음 [S57]), MapTiler(비상업 한정·세션 한도 [S55]). 근거와 재검토 조건은 v0.1 3.7절과 같습니다.

3.6 2026년 정책 변경 이력(교체 가능 구조의 근거)

시점	공급자	변경	근거
2026-01-01	RainViewer	nowcast·위성·색상표 중단, 줌 7 제한, 분당 100회	[S16]

시점	공급자	변경	근거
2026-03-18	OpenSky	ID/PW 인증 종료, OAuth2 전용	[S5]
2026-08경	airplanes.live	비피더 403(제3자 보고)	[S11]
예정	adsb.lol	피더 API 키 요구 예고	[S7]

불확실 — Phase 0에서 직접 확인

- adsb.lol·OpenSky의 한반도 상공 커버리지 비교 자료를 찾지 못했습니다(OpenSky는 자원봉사 수신기망 의존 [S56]). 같은 시각 두 API의 결과 수를 1시간 비교해 1순위를 확정하세요.
- RainViewer 타일의 한국 레이더 포함 여부는 커버리지 마스크 타일(/v2/coverage/...)로 확인하세요 [S17].
- adsb.lol 레이트리밋은 README(없음 [S7])와 제3자 관측(동적 제한 [S11])이 다릅니다. 429 처리를 전제로 설계했습니다.

4. 시스템 아키텍처

4.1 설계 원칙

1. **단일 공개 진입점** — 브라우저는 edge(nginx)만 본다. api·collector·db·redis는 내부 네트워크.
2. **읽기와 수집의 분리** — 외부 API를 부르는 프로세스(collector)와 사용자 요청을 받는 프로세스(api)를 분리해, 외부 지연·장애가 사용자 경로에 번지지 않게 한다.
3. **스트림이 경계** — collector→api는 Redis Streams(소비자 그룹, at-least-once)로만 연결한다. 두 언어는 스키마 파일로 계약한다.
4. **핫 상태는 메모리, 이력은 DB** — 최신 스냅샷·인덱스는 api 메모리에, 이력·통계·감사는 PostGIS에. Redis는 스트림·예산·세션·짧은 캐시.
5. **모든 값에 출처와 시각** — 추정은 추정으로 표시하고, 모르면 비운다.
6. **측정 후 최적화** — 캐시·인덱스·워커 수는 예산 대비 측정값으로 정한다(7장).

4.2 컨텍스트 다이어그램



그림 1. 컨텍스트. 외부 데이터 호출은 전부 서버(collector)가 하고, 브라우저는 정적 타일만 직접 받는다.

4.3 컨테이너 다이어그램 (로컬 Compose)

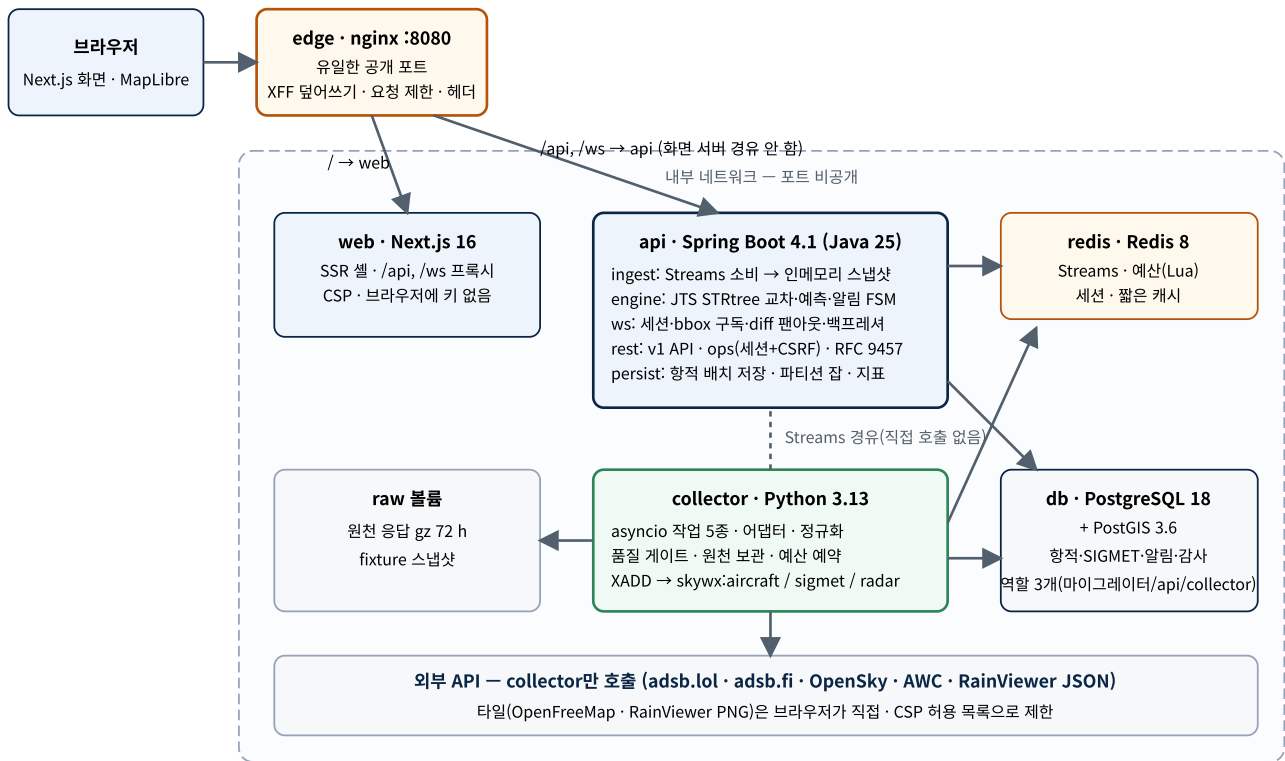


그림 2. 컨테이너 구성. web은 화면과 프록시 설정만 가지며 API 요청은 edge에서 api로 직접 라우팅한다(화면 서버를 거치지 않아 처리량 손실 없음).

4.4 컴포넌트 책임과 경계

컨테이너	컴포넌트	책임	하지 않는 일
collector (Python)	scheduler	작업 5종(region 10 s, global 120 s, sigmet 300 s, radar 60 s, airport 1 d)의 주기 실행, 예산 예약, 지터	DB 도메인 테이블 쓰기
	providers	어댑터·정규화·백오프·폴백 순서	공간 연산
	quality	게이트 규칙, 격리 집계, 원천 보관	—
	publisher	스냅샷 배치를 gzip JSON으로 XADD, MAXLEN 정리, ingest_run·provider_call 기록(수집 전용 DB 역할)	—
api (Java)	ingest	소비자 그룹 XREADGROUP → 스키마 검증 → 인메모리 스냅샷 갱신 → XACK	외부 API 호출
	engine	SIGMET STRtree 재구축(갱신 시), 교차-예측 판정, 알림 FSM, 이벤트 발행	I/O
	ws	세션 관리, bbox 구독, diff 계산, 백프레서(드롭·강제 재동기), ping	도메인 판단
	rest	v1 공개 API, ops API(세션+CSRF+역할), ETag, 요청 제한 확인, RFC 9457	—
	persist	항적·SIGMET·알림·METAR 배치 저장(가상 스레드), 파티션 생성·삭제 잡, 통계 집계	—
web (Next.js)	app	화면·상태·지도 레이어·보간·재생, 서버 컴포넌트 셀, CSP	외부 데이터 API 직접 호출(타일 제외)

컨테이너	컴포넌트	책임	하지 않는 일
edge (nginx)	—	공개 포트, TLS 종단(배포), XFF 덮어쓰기, IP당 요청 제한 (1차), 정적 캐시 헤더, 보안 헤더	애플리케이션 판단

4.5 데이터 흐름 — 수집 → 팬아웃 시퀀스

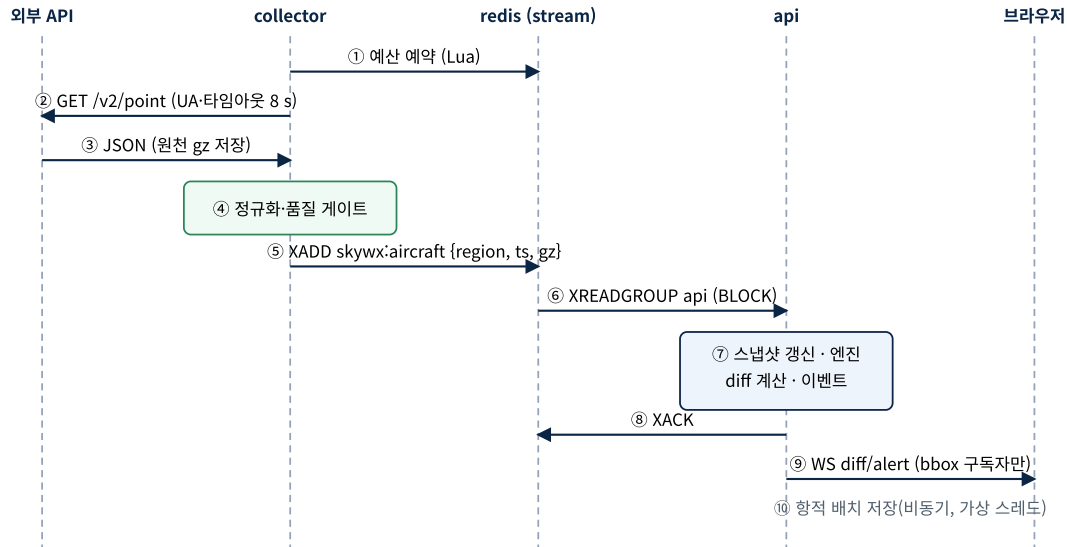


그림 3. 관심 지역 1주기. ⑦이 끝난 뒤에 ⑧ XACK 하므로 api가 죽어도 메시지는 PEL에 남아 재처리된다(at-least-once, 5.3절).

Redis Streams는 append-only 로그처럼 동작하는 자료구조이며 [S62], XREADGROUP은 소비자 그룹을 지원하는 XREAD의 특수 버전으로 [S63] 확인(XACK) 전까지 메시지가 대기 목록에 남습니다. 스트림 3개(skywx:aircraft , skywx:sigmet , skywx:radar)는 MAXLEN ~ 200 으로 잘라 메모리를 제한합니다.

4.6 데이터 흐름 표

데이터	주기	스트림/저장	소비 후 동작
관심 지역 항공기	10 s	skywx:aircraft (region)	스냅샷 갱신 → 엔진 → diff 팬아웃 → 항적 저장(10 s 해상도)
전세계 항공기	120 s	skywx:aircraft (global)	저해상 스냅샷 → 줌 ≤ 5 구독자 → 항적 저장(120 s 해상도)
SIGMET	300 s	skywx:sigmet → sigmet 테이블	STRtree 재구축 → 전체 재판정 → sigmets 이벤트
레이더 프레임	60 s	skywx:radar → Redis 캐시	radar 이벤트(프레임 목록만)
METAR/TAF	요청 시(캐시 5/10분)	api가 collector에 요청? → 아니요: collector가 관심 공항 목록을 10분마다 선제 수집	metar_obs 저장, 카테고리 계산
공항 목록	1일	airport 테이블	지도 공항 레이어

결정 — 요청 시 외부 호출 금지 (ADR-006)

사용자 요청을 처리하는 api는 외부 API를 절대 직접 부르지 않습니다. METAR/TAF도 collector가 관심 지역 공항 (기본 30곳)을 선제 수집합니다. 사용자 트래픽이 외부 한도를 소모하는 경로를 구조적으로 없애기 위해서입니다 (US-21).

5. 신뢰성 설계

5.1 장애 모델

장애	영향	설계 대응
공급자 1곳 5xx/429/타임아웃	해당 데이터 지연	지수 백오프(2→300 s, 지터) → 3회 연속 실패 시 다음 순위 어댑터 → 10분 후 복귀 시도. 모든 전환은 <code>provider_switch</code> 이벤트로 기록
공급자 전부 실패	신규 데이터 없음	api는 마지막 스냅샷 유지 + <code>stale</code> 플래그 + lag 배지. 화면은 비지 않음(FR-19)
collector 프로세스 종료	수집 중단	<code>Compose restart: unless-stopped</code> , 예약된 예산은 TTL로 자동 반환, 재시작 시 현재 시각부터 재개 (과거 재수집 없음)
api 프로세스 종료	WS 끊김, 스냅샷 소실	재시작 시 Redis 스트림의 최근 엔트리와 PEL로 상태 재구성(≤ 60 s), 브라우저는 지수 백오프 재접속 후 snapshot 재수신
Redis 종료	스트림·예산·세션 소실	AOF(everysec) + 볼륨. collector는 XADD 실패 시 로컬 큐(최대 1,000건)에 보관 후 재전송. api는 Redis 미가용 시 마지막 메모리 상태로 서비스
DB 종료·느림	이력 저장 지연	저장은 비동기 배치 큐(메모리 상한 50,000행, 초과 시 오래된 것부터 드롭 + 지표). 조회 API는 503 + Retry-After. 실시간 경로는 DB에 의존하지 않음
WS 클라이언트 느림(백프레서)	서버 메모리 증가	세션별 송신 버퍼 상한(256 KB)·시간 상한(5 s) 초과 시 diff 누적을 버리고 “재동기 필요” 플래그 → 다음 주기 전체 스냅샷. 상한 재초과 시 연결 종료(1008)
스키마 불일치(Python↔Java)	파싱 실패	스트림 메시지에 <code>schema_version</code> . 검증 실패는 DLQ 스트림(<code>skywx:dlq</code>)으로 이동, 운영 화면에 표시. CI 계약 테스트로 사전 차단
시계 불일치	lag·유효시간 오판	모든 시각은 UTC, 수신 시각은 서버 시계 기준. 공급자 시각이 서버보다 30 s 이상 미래면 게이트 격리

5.2 백프레셔와 흐름 제어

- **수집 → 스트림:** 스트림 길이 `MAXLEN ~ 200`. api가 뒤처지면 오래된 스냅샷이 잘리는데, 스냅샷은 “최신만 의미”이므로 손실이 아니라 정상 동작이다. 잘린 건수는 지표로 노출.
- **스트림 → api:** 단일 소비자 그룹 `api`, 소비자 1개(로컬). 처리 순서: 스키마 검증 → 스냅샷 교체(불변 맵 교체, 락 없음) → 엔진 → 이벤트 큐 → XACK.
- **api → 브라우저:** 세션마다 `ConcurrentWebSocketSessionDecorator` 로 감싸 동시 전송을 직렬화한다. JSR-356 세션은 동시 전송을 허용하지 않기 때문이다 [567]. 버퍼·시간 상한 초과 정책은 5.1 참조.

- **브라우저:** 보간 타이머는 250 ms, 탭이 숨겨지면(`visibilitychange`) 구독을 일시 중지해 서버 부하를 줄인다.

5.3 재시작 안전성과 멍등성

- 스트림 메시지 처리 후 XACK(at-least-once). 중복 처리는 스냅샷 교체가 멍등이므로 무해하고, 알림 FSM은 (hex, sigmet_id) 상태를 보고 판단하므로 중복 이벤트가 생기지 않는다.
- 항적 저장은 (hex, ts) 기본키에 ON CONFLICT DO NOTHING . 재처리로 같은 행이 와도 중복되지 않는다.
- SIGMET은 (fir_id, series_id, valid_from) 자연키로 upsert. 같은 경보가 두 번 수신되어도(실제 응답에서 관측됨 [s13]) 한 행이다.
- Graceful shutdown: SIGTERM 수신 시 WS에 going_away (1001) 전송 → 저장 큐 flush(최대 10 s) → 종료.

6. 보안 설계

6.1 보호 대상과 위협 모델(STRIDE)

보호 대상: ① 외부 공급자 자격증명(OpenSky client secret, 기상청 키) ② 외부 호출 예산(남용으로 소진되면 서비스 정지) ③ 운영 기능(설정 변경·로그) ④ 서비스 가용성 ⑤ 데이터 정직성(위·변조된 값이 화면에 오르지 않을 것).

개인정보는 수집하지 않는다(운영자 계정 1개 제외).

STRIDE	위협 시나리오	통제	검증
Spoofing	운영 화면 로그인 무차별 대입, 세션 탈취	BCrypt·계정 잠금 (5회/15분)·HttpOnly·SameSite=Strict·Secure 쿠키, 로그인 시 세션 ID 교체, Redis 세션 TTL 8 h	통합 테스트
Spoofing	X-Forwarded-For 위조로 IP 제한 우회	edge가 XFF를 실제 접속 주소로 덮어쓰고 api는 edge의 고정 IP만 신뢰(포트폴리오 3곳에서 실제 재현한 취약점)	E2E 위조 헤더 3경로
Tampering	WS 메시지·bbox 조작으로 과다 구독	bbox 면적 상한(전세계 뷰는 ≤ 5 만), 연결당 구독 1개, 메시지 크기 ≤ 4 KB, JSON 스키마 검증	통합 테스트
Tampering	외부 응답 조작(중간자)	HTTPS만, 인증서 검증, 원천 보관으로 사후 대조	설정 검사
Repudiation	운영 설정 변경 추적 불가	audit_log (누가·언제·무엇·이전값·이후값), 추가 전용 권한	DB 권한 테스트
Information disclosure	오류 응답·로그에 비밀값·스택 노출	RFC 9457 형식으로 내부 정보 제거 [s69], 로그 마스킹 10패턴, 비인가 ops 경로는 404	단위·통합
Information disclosure	브라우저 번들에 키 포함	브라우저는 키를 갖지 않음(BFF), NEXT_PUBLIC_ 접두사 검사, gitleaks	CI
Denial of service	공개 API·WS 폭주로 외부 예산 소진	사용자 트래픽과 외부 호출의 구조적 분리(ADR-006), IP당 분당 120회(edge 1차 + api Lua 2차), WS 200연결·IP당 5	k6
Denial of service	거대 SIGMET 폴리곤·좌표 폭탄	폴리곤 점 $\leq 2,000$, 면적 상한, 파싱 타임아웃	fixture
Elevation of privilege	api 계정으로 DDL·감사 로그 삭제	DB 역할 3개(마이그레이터/api/collector), api는 DML만, audit_log는 INSERT만	권한 테스트
Elevation	컨테이너 탈출·취약 이미지	비root 실행, read-only 루트 FS, cap_drop ALL, Trivy 스캔	CI

6.2 총별 통제

층	통제
edge	단일 공개 포트, limit_req (IP당 10 r/s burst 30), 보안 헤더(HSTS는 배포 시), 요청 본문 1 MB, proxy_set_header X-Forwarded-For \$remote_addr (뒤어쓰기), 서버 토큰 숨김
web	CSP: default-src 'self'; connect-src 'self' https://tiles.openfreemap.org https://tilecache.rainviewer.com https://api.rainviewer.com; img-src 'self' data: blob: https://tiles.openfreemap.org https://tilecache.rainviewer.com; worker-src 'self' blob: ; X-Frame-Options DENY, Referrer-Policy strict-origin-when-cross-origin. 외부 스크립트 없음(MapLibre는 번들)
api	Spring Security: 공개 경로 허용, /api/v1/ops/** 는 ROLE_OPS + 세션 + CSRF 이중 제출, 실패는 404. 입력 검증 (Bean Validation), bbox·시간 범위·페이지 크기 상한. RFC 9457 오류. 요청 추적 ID(응답 헤더·로그). Actuator는 별도 포트(내부)
collector	외부 키는 이 프로세스에만. 응답 크기 상한 20 MB, 타임아웃 8 s, 리다이렉트 금지, 허용 호스트 목록 외 호출 금지(SSRF 방지)
redis	내부 네트워크만, requirepass , 위험 명령 rename(FLUSHALL 등), maxmemory 512 MB + allkeys-lru는 캐시 DB에만(스트림 DB 분리: db0 스트림·예산, db1 캐시·세션)
db	역할 3개, skywx_api 는 DML만, skywx_collector 는 ingest 테이블 INSERT만, 파티션 DDL은 마이그레이터가 미리 생성(30일치)
CI	gitleaks(전체 이력), CodeQL(Java·JS·Python), Trivy(이미지), npm audit·pip-audit, Dependabot 주간, 테스트 통과 없이 병합 불가

6.3 운영자 인증 흐름

1. POST /api/v1/ops/session (아이디·비밀번호) → BCrypt 검증 → 세션 ID 재발급 → Redis 세션(TTL 8 h) → Set-Cookie: SKYWX_SESSION; HttpOnly; SameSite=Strict + CSRF 토큰 쿠키.
2. 변경 요청은 X-CSRF-Token 헤더 필수. 실패 시 403, 비인증은 404.
3. 운영자 계정은 초기 마이그레이션이 아니라 make ops-user 명령으로 생성(비밀번호는 환경변수/프롬프트, 소스에 없음).
4. 모든 ops 변경은 audit_log 에 기록.

7. 성능 설계

7.1 성능 예산

경로	예산(p95)	측정 조건
수집 수신 → WS 반영(관심 지역)	≤ 1.5 s	메시지 내 fetched_at 대비 브라우저 수신 시각
엔진 1주기(스냅샷 갱신+판정+diff)	≤ 50 ms(관심 지역 500대), ≤ 300 ms(전세계 10k대)	Micrometer 타이머
REST 스냅샷/SIGMET	≤ 80 ms 적중 / ≤ 300 ms 미스	k6 100 rps, 3분
REST 항적·재생	≤ 100 ms / ≤ 300 ms	PostGIS 인덱스 사용 확인(EXPLAIN)
WS 팬아웃 200연결	≤ 500 ms, api 힙 ≤ 512 MB	k6 websockets, JFR

경로	예산(p95)	측정 조건
프론트	3,000대 $\geq$ 30 fps, 첫 JS $\leq$ 400 KB gzip	DevTools, Lighthouse

7.2 용량 산정(추정 — 구현 후 실측으로 교체)

항목	계산	결과
관심 지역 항공기 수	가정 300~800대(시간대별)	500대 기준
전세계 항공기 수	가정 8,000~12,000대	10,000대 기준
스냅샷 메시지 크기	1대 $\approx$ 160 B JSON $\rightarrow$ 500대 80 KB, 10k대 1.6 MB $\rightarrow$ gzip $\approx$ 1/5	16 KB / 320 KB
WS diff(지역)	변경 비율 90% $\times$ 500 $\times$ 120 B / 10 s	$\approx$ 5.4 KB/s/클라이언트, 200명 $\approx$ 1.1 MB/s
항적 저장 행/초	500/10 s + 10,000/120 s	$\approx$ 133 행/s $\approx$ 1,150만 행/일
항적 저장 용량	행 $\approx$ 70 B + 인덱스 $\approx$ 60 B	$\approx$ 1.5 GB/일 $\rightarrow$ 72 h $\approx$ 4.5 GB(로컬 상한 10 GB 내)
1분 요약	10,500대 $\times$ 1,440분 $\times$ 70 B	$\approx$ 1 GB/일 $\rightarrow$ 30일 30 GB로 초과 $\rightarrow$ 요약은 관심 지역만(500대: 50 MB/일, 30일 1.5 GB)
Redis 메모리	스트림 200 $\times$ 320 KB + 캐시	$\leq$ 128 MB
api 힙	스냅샷 2벌 $\times$ 10k $\times$ 1 KB 객체 + 인덱스 + WS 버퍼 200 $\times$ 256 KB	$\approx$ 120 MB, 상한 512 MB

결정 — 보존 정책 (ADR-007)

원해상도 항적 72시간(전 항공기), 1분 요약 30일(관심 지역만), 알림·SIGMET·통계는 영구. 요약은 매시 배치가 track_point 에서 track_point_1m 으로 생성한다. 로컬 디스크 10 GB 상한을 넘지 않도록 파티션 삭제 잡이 매일 03:00에 돈다.

7.3 성능 전략

- **불변 스냅샷 교체:** 스냅샷은 `Map<hex, State>` 를 통째로 교체(참조 교체)해 읽기 락이 없다. diff는 이전/현재 맵 비교 $O(N)$.
- **bbox 구독 필터:** 세션마다 bbox를 두고 diff 계산 전에 공간 필터. 전세계 뷰(줌 $\leq$ 5)는 별도 저해상 스냅샷(소수점 2자리, 필드 6개)만 전송.
- **엔진:** JTS STRtree는 질의 전용 R-tree로 [s68] SIGMET 갱신 시에만 재구축하고, 질의는 스레드 안전하므로 판정은 병렬 스트림으로 처리한다.
- **가상 스레드:** Spring Boot의 `spring.threads.virtual.enabled=true` (Java 21+) [s66]로 저장·조회 블로킹 I/O를 가상 스레드에서 처리한다.
- **DB:** 항적은 일 파티션 + (hex, ts) PK + BRIN(ts). 재생 질의는 파티션 프루닝 + `DISTINCT ON (hex) ... ORDER BY hex, ts DESC`. PostGIS `ST_Intersects` 는 경계상자 비교로 공간 인덱스를 자동 활용한다 [s71].
- **HTTP:** ETag(스냅샷 버전) + `Cache-Control: max-age=5`, gzip/zstd(edge), 정적 파일 1년 immutable.
- **프론트:** 심볼 레이어 1개 + GeoJSON `setData` 250 ms, 뷰포트 밖 제외, Web Worker에서 보간 계산(메인 스레드 보호).

7.4 측정 계획

도구	무엇을	언제
k6 (HTTP·websockets) [S73]	REST 100 rps, WS 200 연결 램프업, 한도 초과 시나리오 (429·Retry-After)	Phase 3 말, 주요 변경 후
Micrometer → Prometheus	엔진 시간, 스트림 지연, WS 세션 수·드롭 수, 공급자 호출·예산, DB 큐 길이	상시
JFR	부하 중 힙·GC·스레드 프로파일	병목 조사 시
EXPLAIN (ANALYZE, BUFFERS)	재생·항적·통계 질의 계획, 인덱스 사용	질의 추가·변경 시
Chrome DevTools · Lighthouse	fps, 메인 스레드 시간, LCP, 전송량	화면 변경 시

8. 데이터 모델

8.1 ERD

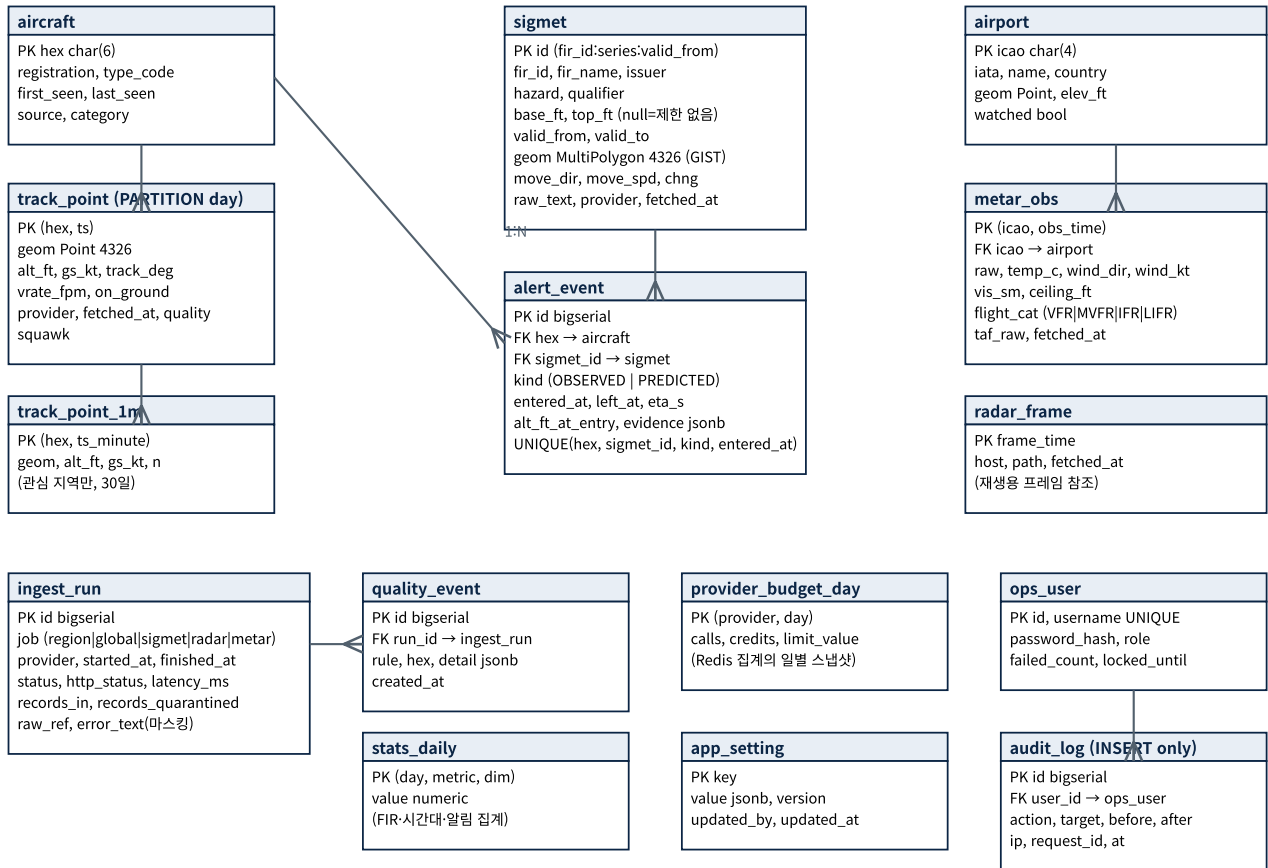


그림 4. ERD(주요 열만). 실시간 스냅샷은 DB에 없고 api 메모리에만 있다. 색이 진한 머리글은 도메인 테이블, 아래 줄은 운영·감사 테이블이다.

8.2 테이블 정의 요약

테이블	쓰기 주체	핵심 규칙	보존
aircraft	api	첫 관측 시 upsert(등록·기종은 공급자 값), last_seen 갱신은 1분에 1회로 제한	영구
track_point	api	일 파티션, PK(hex, ts), BRIN(ts), GIST(geom)는 두지 않음(재생은 시간 → hex 순 조회로 충분, 인덱스 비용 절감). quality 는 게이트 통과 등급	72 h(파티션 DROP)
track_point_1m	api(배치)	매시 집계, 관심 지역 bbox 내만	30일
sigmet	api	자연키 upsert, MultiPolygon(GIST), base null→0, top null→NULL(상한 없음), 원문 보존	영구
alert_event	api	kind OBSERVED/PREDICTED 분리, evidence에 판정 근거(폴리곤 id·고도·시각·거리)	영구
airport	collector	AWC airport 데이터에서 일 1회, watched 30곳이 METAR 선제 수집 대상	영구

테이블	쓰기 주체	핵심 규칙	보존
metar_obs	collector	PK(icao, obs_time), flight_cat은 저장 시 계산	30일
radar_frame	collector	프레임 시각·경로만(타일은 저장하지 않음)	7일
ingest_run / quality_event	collector	실행마다 1행, 오류 원문은 마스킹 후 저장, raw_ref로 원천 추적	30일
provider_budget_ day	collector	Redis 예산 카운터의 일별 스냅샷(자정)	영구
ops_user / audit_log / app_setting	api	audit_log는 INSERT만(역할 권한), app_setting은 version으로 낙관적 잠금	영구
stats_daily	api(배치)	일 1회 03:30 집계, 통계 화면 전용	영구

8.3 DDL 발췌 (Flyway V1)

```

CREATE EXTENSION IF NOT EXISTS postgis;

CREATE TABLE aircraft (
    hex          char(6) PRIMARY KEY,
    registration text, type_code text, category text, source text,
    first_seen   timestampz NOT NULL, last_seen timestampz NOT NULL
);

CREATE TABLE track_point (
    hex char(6) NOT NULL, ts timestampz NOT NULL,
    geom geometry(Point, 4326) NOT NULL,
    alt_ft int, gs_kt real, track_deg real, vrate_fpm real, on_ground bool NOT NULL DEFAULT false,
    squawk char(4), provider text NOT NULL, fetched_at timestampz NOT NULL, quality smallint NOT NULL DEFAULT 0,
    PRIMARY KEY (hex, ts)
) PARTITION BY RANGE (ts);
CREATE INDEX track_point_ts_brin ON track_point USING BRIN (ts);
-- 파티션은 마이그레이터가 30일치 선생성, 삭제 잡은 72 h 초과분 DROP

CREATE TABLE sigmet (
    id text PRIMARY KEY, fir_id text NOT NULL, fir_name text, issuer text,
    hazard text NOT NULL, qualifier text, base_ft int NOT NULL DEFAULT 0, top_ft int,
    valid_from timestampz NOT NULL, valid_to timestampz NOT NULL,
    geom geometry(MultiPolygon, 4326) NOT NULL,
    move_dir text, move_spd text, chng text, raw_text text NOT NULL,
    provider text NOT NULL, fetched_at timestampz NOT NULL,
    CHECK (valid_to > valid_from), CHECK (top_ft IS NULL OR top_ft >= base_ft)
);
CREATE INDEX sigmet_geom_gist ON sigmet USING GIST (geom);
CREATE INDEX sigmet_valid ON sigmet (valid_to, valid_from);

CREATE TABLE alert_event (
    id bigserial PRIMARY KEY,
    hex char(6) NOT NULL REFERENCES aircraft(hex),
    sigmet_id text NOT NULL REFERENCES sigmet(id),
    kind text NOT NULL CHECK (kind IN ('OBSERVED', 'PREDICTED')),
    entered_at timestampz NOT NULL, left_at timestampz, eta_s int,
    alt_ft_at_entry int, evidence jsonb NOT NULL,
    UNIQUE (hex, sigmet_id, kind, entered_at)
);

CREATE TABLE audit_log (

```

```

id bigserial PRIMARY KEY, user_id int REFERENCES ops_user(id),
action text NOT NULL, target text, before jsonb, after jsonb,
ip inet, request_id text, at timestampz NOT NULL DEFAULT now()
);
GRANT INSERT ON audit_log TO skywx_api;  -- UPDATE/DELETE 없음

```

8.4 재생 질의 예

```

-- 시각 :t 기준 bbox 안 항공기의 마지막 위치 (파티션 프루닝 + PK 역순)
SELECT DISTINCT ON (hex) hex, ts, ST_X(geom) lon, ST_Y(geom) lat, alt_ft, gs_kt, track_deg
FROM track_point
WHERE ts BETWEEN :t - interval '3 minutes' AND :t
      AND geom && ST_MakeEnvelope(:lomin, :lamin, :lomax, :lamax, 4326)
ORDER BY hex, ts DESC;

```

9. API 명세

9.1 공통 규약

항목	규약
기본 경로·버전	/api/v1/... . 호환성 깨지는 변경은 v2로. 응답 JSON은 snake_case
시각	모두 UTC ISO-8601(2026-09-27T05:10:00Z). Unix 초는 WS 내부 메시지에만
단위	고도 ft, 속도 kt, 수직속도 ft/min, 거리 NM, 좌표 WGS84 (lon, lat 순은 GeoJSON, 쿼리 파라미터는 lat/lon 명시)
출처 메타	모든 데이터 응답에 meta: {provider, fetched_at, lag_s, stale, generated_at, request_id}
캐시	ETag (스냅샷 버전) + Cache-Control: public, max-age=N . 조건부 요청 시 304
요청 제한	IP당 분당 120회. 응답 헤더 X-RateLimit-Limit / Remaining / Reset , 초과 시 429 + Retry-After
오류	RFC 9457 application/problem+json [S69]: type, title, status, detail, instance + 확장 code, request_id . 내부 정보 없음
페이지네이션	커서 방식(cursor , limit ≤ 200), 시간 범위 상한 24 h
인증	공개 API 없음. /api/v1/ops/** 는 세션 쿠키 + CSRF 헤더 + ROLE_OPS, 비인가는 404
추적	요청마다 X-Request-Id 발급·에코, 로그·오류 본문에 동일 값

9.2 REST v1 엔드포인트

경로	설명·파라미터	응답	캐시
항공기			
GET /aircraft? bbox=lomin,lamin,lomax,lamax&detail=lite full	현재 스냅샷. bbox 필수, 면적 ≤ 2,500 sq°(초과 시 422). lite는 6필드	GeoJSON FeatureCollection + meta	5 s
GET /aircraft/{hex}	상세(정적 정보 + 현재 상태 + 활성 알림)	AircraftDetail	5 s
GET /aircraft/{hex}/track?from&to	항적(기본 최근 2 h, 최대 24 h), 다운샘플 step_s	GeoJSON LineString + points[]	30 s
GET /aircraft/search? q=callsign reg hex	접두 검색(현재 스냅샷 + aircraft 테이블), ≤ 20건	[]AircraftLite	5 s
기상·경보			
GET /sigmets? active=true&bbox=&hazard=	SIGMET GeoJSON(속성: hazard, qualifier, base_ft, top_ft, valid_from/to, fir_id, raw_text)	FeatureCollection	60 s
GET /sigmets/{id}	단건 + 현재 안에 있는 항공기 hex 목록	SigmetDetail	10 s
GET /alerts? kind=observed predicted&active=true	알림 목록(근거 포함)	[]Alert	5 s
GET /alerts/history? from&to&hex&cursor	알림 이력(커서)	Page<Alert>	60 s
GET /radar/frames	RainViewer 프레임 목록(host, past[]) 전달	RadarFrames	60 s

경로	설명·파라미터	응답	캐시
GET /airports?bbox=&watched=true	공항 + 최신 METAR 요약(flight_cat)	FeatureCollection	60 s
GET /airports/{icao}/wx	METAR 최신·TAF 원문·해석·카테고리	AirportWx	300 s
이력·통계			
GET /replay?at=&bbox=	시각 at 의 항공기 스냅샷(3분 창) + 그때 유효했던 SIGMET	ReplayFrame	3600 s
GET /stats/sigmet? from&to&group=fir hazard	SIGMET 집계	[]StatRow	600 s
GET /stats/traffic?day=®ion=	시간대별 항공기 수	[]StatRow	600 s
GET /stats/alerts?from&to	알림 건수·평균 체류 시간	[]StatRow	600 s
시스템			
GET /status	공개 상태: 공급자별 lag·stale, 활성 공급자, 스냅샷 버전(비밀값·수치 예산 없음)	PublicStatus	5 s
GET /healthz (edge에서만 노출)	liveness/readiness(Redis·DB·스트림 소비 지연)	{status}	—
운영(인증 필요, 비인가 404)			
POST /ops/session · DELETE /ops/session	로그인·로그아웃	204	—
GET /ops/providers	공급자별 호출량·예산 잔량·마지막 성공/실패·연속 실패·전환 이력	[]ProviderStatus	—
GET /ops/runs?job=&status=&cursor	ingest_run 목록(오류 원문 마스킹)	Page<IngestRun>	—
GET /ops/quality?from&to	규칙별 격리 건수, 최근 사례	QualitySummary	—
GET /ops/settings · PUT /ops/settings/{key}	런타임 설정(폴링 주기·공급자 순서·관심 지역·watched 공항). PUT은 If-Match: version 필수	Setting	—
POST /ops/providers/{name}/disable enable	수동 전환(감사 기록)	204	—
GET /ops/audit?cursor	감사 로그	Page<Audit>	—

9.3 스키마 (발체)

```
// AircraftState (스트림·REST·WS 공통, schemas/aircraft_state.v1.json)
{
  "hex": "71c0a1", "callsign": "KAL081", "registration": "HL8001", "type_code": "B77W",
  "lat": 37.4602, "lon": 126.4407, "alt_ft": 35000, "gs_kt": 470.0, "track_deg": 82.5,
  "vrate_fpm": 0, "on_ground": false, "squawk": "1234",
  "seen_at": "2026-09-27T05:10:03Z", "provider": "adsb_lol", "fetched_at": "2026-09-27T05:10:05Z",
  "quality": 0, // 0 통과, 1 경고(보간 어려움), 2 격리(전송 안 함)
  "estimated": false // 서버 값은 항상 false, 브라우저 보간 결과만 true
}

// Alert
{
  "id": 9127, "kind": "PREDICTED", "hex": "71c0a1", "callsign": "KAL081",
  "sigmet_id": "RKRR:F02:1790485200", "hazard": "ICE", "qualifier": "SEV",
  "entered_at": null, "eta_s": 240, "alt_ft": 17000,
  "evidence": { "polygon_index": 0, "band_ft": [14000, 21000], "valid_to": "2026-09-27T09:00:00Z",
    "method": "dead_reckoning_10min", "distance_nm": 31.2 },
}
```

```

"meta": { "generated_at": "2026-09-27T05:10:06Z", "request_id": "01J..." }
}

// RFC 9457 오류
HTTP/1.1 422 Unprocessable Content
Content-Type: application/problem+json
{ "type": "https://skywx.dev/problems/bbox-too-large", "title": "bbox too large",
  "status": 422, "detail": "bbox area 3200 sq° exceeds 2500", "instance": "/api/v1/aircraft",
  "code": "BBOX_TOO_LARGE", "request_id": "01J..." }

```

9.4 OpenAPI 발췌 (springdoc가 생성, 저장소에 스냅샷 커밋)

```

openapi: 3.1.0
info: { title: SkyWx API, version: "1.0.0" }
paths:
  /api/v1/aircraft:
    get:
      summary: 현재 항공기 스냅샷
      parameters:
        - { name: bbox, in: query, required: true, schema: { type: string, pattern: '^-?\d+(\.\d+)?(-?\d+(\.\d+)?){3}$' } }
        - { name: detail, in: query, schema: { type: string, enum: [lite, full], default: lite } }
      responses:
        "200": { description: OK, headers: { ETag: { schema: { type: string } }, X-RateLimit-Remaining: { schema: { type: integer } } },
              content: { application/geo+json: { schema: { $ref: '#/components/schemas/AircraftCollection' } } } }
        "304": { description: Not Modified }
        "422": { $ref: '#/components/responses/Problem' }
        "429": { $ref: '#/components/responses/Problem' }

```

9.5 WebSocket 프로토콜 v1 — /ws/v1

방향	type	내용·규칙
C→S	hello	{proto:1, client:"web/1.0"} . 첫 메시지. 5 s 내 미수신 시 종료(1002)
C→S	subscribe	{bbox:[lomin,lamin,lomax,lamax], zoom:7, detail:"lite"} . 연결당 1개, 재전송 시 교체. bbox 면적 > 2,500 sq°이면 zoom ≤ 5 일 때만 허용(저해상 전세계)
C→S	select	{hex} 선택 항공기(고빈도 상세·항적 스트림), null로 해제
C→S	pause / resume	탭 숨김 시 전송 중지
S→C	welcome	{session_id, server_time, snapshot_version, limits:{max_bbox_area, diff_interval_s}}
S→C	snapshot	{v, ts, aircraft:[AircraftState...], sigmets_version} 구독 직후·30 s마다·재동기 시
S→C	diff	{v, ts, upsert:[...], remove:["hex"...]} 10 s(지역)/120 s(전세계). v 는 단조 증가, 건너뛴 v를 받으면 클라이언트가 resync 요청
S→C	sigmets	SIGMET GeoJSON 전체(갱신 시)
S→C	alert	{event:"ENTERED" "LEFT" "PREDICTED" "PREDICTION_CLEARED", alert:Alert}
S→C	radar	프레임 목록 갱신
S→C	status	{providers:{aircraft:"adsb_lol", lag_s:3, stale:false}, ...} 30 s마다·변경 시
S→C	error	RFC 9457과 같은 필드. 치명적이면 close code 1008(정책)·1013(과부하)

방향	type	내용·규칙
양방향	ping/pong	프로토콜 ping 30 s, 2회 무응답 시 종료

- 메시지는 텍스트 JSON, 1 MB 상한(서버→클라이언트), 4 KB 상한(클라이언트→서버). 압축은 permessage-deflate(edge에서 협상).
- 연결 상한: 전체 200, IP당 5. 초과 시 1013으로 즉시 종료.
- 서버 구현은 Spring의 `TextWebSocketHandler` 확장 [S67], 세션은 `ConcurrentWebSocketSessionDecorator(session, 5000 ms, 262144 B)` 로 감싼다.

10. 핵심 알고리즘

10.1 클라이언트 보간 (dead reckoning)

구면 위 “출발점·방위·거리 → 도착점” 공식을 쓴다 [S37]. 1 kt = 1,852 m/h [S38].

```

φ2 = asin( sin φ1 · cos δ + cos φ1 · sin δ · cos θ )
λ2 = λ1 + atan2( sin θ · sin δ · cos φ1, cos δ - sin φ1 · sin φ2 )    δ = d / R, R = 6,371 km
d = gs_kt × 1852 × elapsed_s / 3600

// Web Worker에서 250 ms마다 실행, 결과에 estimated=true
function predict(s, nowMs) {
  const dt = (nowMs - s.seenAtMs) / 1000;
  if (s.onGround || s.gsKt == null || s.trackDeg == null || Math.abs(s.lat) > 85) return { ...s, estimated: false };
  const d = s.gsKt * 1852 * dt / 3600, δ = d / 6371000, θ = rad(s.trackDeg), φ1 = rad(s.lat), λ1 = rad(s.lon);
  const φ2 = Math.asin(Math.sin(φ1)*Math.cos(δ) + Math.cos(φ1)*Math.sin(δ)*Math.cos(θ));
  const λ2 = λ1 + Math.atan2(Math.sin(θ)*Math.sin(δ)*Math.cos(φ1), Math.cos(δ) - Math.sin(φ1)*Math.sin(φ2));
  return { ...s, lat: deg(φ2), lon: wrap180(deg(λ2)), altFt: s.altFt + (s.vrateFpm ?? 0) * dt / 60,
    estimated: true, stale: dt > 60 };
}

```

새 서버 값이 오면 예측 위치에서 관측 위치로 500 ms 완화(easing). 60 s 초과는 반투명, 300 s 초과는 제거(서버가 remove 전송).

10.2 교차 판정 (현재)

```

// SIGMET 갱신 시(5분) — STRtree 재구축, 질의는 스레드 안전 [S68]
STRtree index = new STRtree();
for (Sigmet s : active) for (Polygon p : s.polygons()) index.insert(p.getEnvelopeInternal(), new Item(s, p));
index.build();

// 스냅샷마다(10 s) — 병렬 스트림
Map<String, List<Hit>> hits = states.parallelStream().filter(a -> !a.onGround() && a.altFt() != null)
  .flatMap(a -> {
    Point pt = gf.createPoint(new Coordinate(a.lon(), a.lat()));
    return index.query(pt.getEnvelopeInternal()).stream().map(Item.class::cast)
      .filter(it -> it.prepared().intersects(pt)) // PreparedGeometry로 정밀 판정
      .filter(it -> it.sigmet().validAt(now))
      .filter(it -> a.altFt() >= it.sigmet().baseFt() && (it.sigmet().topFt() == null || a.altFt() <=
it.sigmet().topFt()))
      .map(it -> new Hit(a.hex(), it.sigmet().id(), it.index(), a.altFt()));
  }).collect(groupingByConcurrent(Hit::hex));

```

- 복잡도 $\approx O(N \log M)$. 10k $\times$ 200에서 수십 ms(NFR-05, JMH로 확인).
- 화산재(VA)·태풍(TC)은 base가 지표(0)인 경우가 많아 지상 항공기가 잡히므로 on_ground 제외.
- 경계 근처 깜빡임 방지: FSM은 진입을 2회 연속, 이탈을 3회 연속 판정 후 확정.

10.3 진입 예측 (10분 dead reckoning) — 신규

```
for each aircraft a (not on ground, gs  $\geq$  60 kt, track known):
    path = [predict(a, +60s), predict(a, +120s), ..., predict(a, +600s)]    // 10점, 10.1 공식 서버판
    line = LineString(a.pos, path...)
    for item in index.query(line.envelope):
        if !item.prepared.intersects(line): continue
        // 첫 교차 시각: 선분별 교차 검사로 60 s 구간을 찾고 그 안에서 선형 보간
        t_entry = firstIntersectionSeconds(item.polygon, a.pos, path)
        alt_at_entry = a.alt_ft + a.vrate_fpm * t_entry / 60
        if bandContains(item.sigmet, alt_at_entry) and item.sigmet.validAt(now + t_entry):
            emit PREDICTED(hex, sigmet, eta_s = t_entry, evidence = {...})
```

- 예측 알림은 관측 알림과 별도 유형(PREDICTED)이며 화면에서 점선·“추정” 라벨로 구분한다(NFR-16).
- 같은 (hex, sigmet)에 대해 ETA는 갱신만 하고 이벤트는 처음 한 번, 예측이 사라지면 PREDICTION_CLEARED .
- 선회 중이거나 속도 변화가 큰 기체는 오차가 커진다. 최근 3회 관측의 트랙 변화가 15°를 넘으면 예측을 내지 않는다 (문서화된 한계).

10.4 SIGMET 파싱·정제

1. geom:"AREA" 는 링 1개, "AREAS" 는 링 여러 개. 각 링에서 lon/lat 가 null인 점 제거, 점 < 3이면 링 폐기(2026-09-27 실응답에서 관측 [S13]).
2. 첫 점 $\neq$ 끝 점이면 닫고, 자기교차는 buffer(0) 로 정리. 경도 $\pm 180^\circ$ 를 걸치는 링은 두 개로 분할.
3. base null $\rightarrow$ 0(SFC), top null $\rightarrow$ 상한 없음. 단위는 피트.
4. 중복 수신은 자연키 fir_id:series_id:valid_from 로 upsert. 유효시간이 지난 경보는 인덱스에서 제외하되 DB에는 남긴다(재생용).
5. “N OF LINE ...”처럼 폴리곤이 아닌 라인형 서술은 좌표가 2점뿐이라 폴리곤을 만들 수 없다. 이런 경보는 **목록에는 원문으로 표시하되 판정에서는 제외**하고, 제외 사실을 화면에 적는다.

10.5 비행 카테고리 계산

AWC 기준으로 MVFR은 실링 1,000~3,000 ft AGL 및/또는 시정 3~5 mi [S70]이며, 같은 문서에서 VFR(> 3,000 ft-> 5 mi), IFR(500 ft 이상 1,000 ft 미만·1 mi 이상 3 mi 미만), LIFR(< 500 ft-< 1 mi)로 정의됩니다. 실링과 시정 중 **더 나쁜 쪽**이 카테고리를 정하고, 실링이 없으면(SK/CLR) 시정만으로 판단합니다. 경계값 8건을 단위 테스트로 고정합니다.

10.6 diff 계산과 재동기

```
diff(prev, curr, bbox):
    upsert = [s for s in curr if inBbox(s) and (prev[s.hex] is None or changed(prev[s.hex], s))]
    remove = [h for h in prev if inBbox(prev[h]) and (h not in curr or not inBbox(curr[h]))]
    changed(): lat/lon 1e-4°, alt 25 ft, gs 1 kt, track 1°, squawk/on_ground 변화 중 하나라도
```

세션별 lastSentVersion 을 유지하고, 클라이언트가 v 불연속을 감지하면 resync 로 전체 스냅샷을 다시 받는다. 30 s마다 전체 스냅샷을 보내 누적 오차를 지운다.

11. 프론트엔드 설계

11.1 화면 구성

화면	내용	주요 컴포넌트
/ (상황판)	전체 화면 지도 + 레이어 토클 + 우측 패널(알림/상세) + 하단 레이더 타임라인 + 상단 상태 바(출처·지연·공급자)	MapView, LayerPanel, AlertPanel, AircraftCard, RadarTimeline, StatusBar
/replay	시각 슬라이더(1×~60×) + 그때의 항적·SIGMET·레이더 프레임	ReplayControls, MapView(replay 소스)
/stats	FIR별 SIGMET, 시간대별 트래픽, 알람 건수(Recharts)	StatCards, Charts
/airports/[icao]	METAR/TAF 원문·해석·카테고리 이력	WxCard
/ops (로그인)	공급자 상태·예산·실행 이력·품질 게이트·설정·감사 로그	ProviderTable, RunLog, QualitySummary, SettingsForm
/about	데이터 출처·라이선스·한계·설계 문서 링크	—

11.2 지도 레이어 규칙

레이어	소스	규칙
배경	OpenFreeMap dark (기본)/ <code>Liberty</code> [S23]	어두운 배경에서 기상 색이 잘 보임. attribution 유지
레이더	<code>raster 512</code> , <code>maxzoom 7</code> , <code>opacity 0.6</code>	프레임별 소스 사전 생성, <code>raster-opacity</code> 만 전환. 커버리지 밖은 마스크 타일로 회색 표시
SIGMET	<code>geojson fill+line</code>	hazard 색(TS 주황·TURB 보라·ICE 하늘·MTW 회청·VA 회갈·TC 빨강), 채움 0.15, 항공기가 안에 있으면 외곽선 굵게, 만료 30분 전부터 점선
항공기	<code>geojson symbol</code> (자체 SVG 아이콘)	<code>icon-rotate =track</code> , 고도 색 램프, <code>stale 0.4</code> , 비상 <code>squawk</code> 빨간 테두리, <code>estimated</code> 는 아이콘 테두리 점선
예측 궤적	<code>geojson line</code> (점선)	선택 항공기 또는 예측 알람 대상만 10분 궤적
항적	<code>geojson line</code>	선택 항공기 2 h, 고도 그라데이션
공항	<code>geojson circle</code>	중 ≥ 6 , 카테고리 색(VFR 초록·MVFR 파랑·IFR 빨강·LIFR 자주), 클릭 시 WxCard

11.3 근거 카드(알림 상세)

알림 하나를 펼치면 “어느 경보(FIR·번호·hazard) · 어느 고도대(base-top) · 항공기 고도 · 유효시간 · 판정 시각 · 관측/추정 여부 · 예측이면 ETA와 방법”을 한 카드로 보여 준다. 근거 없는 알람은 서버가 생성하지 못한다(`evidence NOT NULL`).

11.4 상태·데이터 흐름

- 서버 데이터(스냅샷·SIGMET·알림)는 `useSyncExternalStore` 기반 스토어(리렌더 최소화), UI 상태는 Zustand.
- WS 클라이언트는 지수 백오프 재접속(1→30 s), 재접속 후 `subscribe · select` 재전송, v 불연속 시 `resync`.
- 보간은 Web Worker에서 250 ms, 결과만 메인 스레드로 전달해 `setData`.

- Next.js 16은 Turbopack이 기본이며 Node.js 20.9 이상이 필요하다 [S72]. 서버 컴포넌트는 셀·about·stats 초기 데이터에만 쓰고 지도는 클라이언트 컴포넌트.
- react-map-gl v8.1은 MapLibre GL JS v4·v5·v6을 지원하며 [S61] v5로 시작한다.

12. 기술 스택과 ADR

12.1 권장 스택

계층	선택	이유	근거
edge	nginx 1.27(공식 이미지)	단일 진입·XFF 덮어쓰기·요청 제한·헤더. 포트폴리오 3곳과 같은 패턴이라 실수 재발 방지	—
web	Next.js 16 · React 19 · TypeScript · Tailwind 4 · MapLibre GL JS 5 + react-map-gl/maplibre · Recharts	BFF 프록시·CSP 패턴 재사용, Turbopack 기본, Node 20.9+ [S72]; MapLibre BSD-3 [S22]	[S22][S25][S61][S72]
JS 런타임	Node.js 24 (Active LTS)	2026-09 Active LTS [S27], 프로덕션은 LTS만 [S28]	[S27][S28]
api	Java 25 · Spring Boot 4.1 · Spring MVC(가상 스레드) · Spring WebSocket · Spring Security · Spring Session(Redis) · JdbcClient · Flyway · springdoc · JTS · Lettuce · Micrometer	Java 25는 2025-09-16 GA [S75], Oracle 장기 지원 8년 이상 [S64]; Spring Boot 4.1.1(2026-08) [S65]; 가상 스레드 [S66]; WebSocket 세션 직렬화 [S67]; JTS STRtree [S68]	[S64]–[S68][S75]
collector	Python 3.13 · uv · asyncio · httpx · pydantic v2 · shapely(검증) · redis-py	3.13은 유지보수 라인 [S31]; uv 단일 도구 [S33]; httpx 비동기 [S35]	[S31][S33][S35]
db	PostgreSQL 18 + PostGIS 3.6	파티션·BRIN·GIST, ST_Intersects 인덱스 활용 [S71]. Apple Silicon 이미지 주의(13.4)	[S39]–[S41][S71]
redis	Redis 8(AGPLv3 선택)	Streams 소비자 그룹 [S62][S63], 3중 라이선스 [S42][S58]	[S42][S58][S62][S63]
테스트	JUnit 5 · Testcontainers(PostGIS) · WireMock · JMH · pytest · respx · Vitest · Playwright · k6	실제 DB로 통합 테스트 [S74], WS 부하 [S73]	[S73][S74]
품질·보안	ruff · mypy · ESLint · Prettier · gitleaks · CodeQL · Trivy · Dependabot · pre-commit · ADR	기존 포트폴리오와 동일 파이프라인	—
컨테이너	OrbStack(개인 무료) 또는 Docker Desktop, Docker Compose	[S43][S44]	[S43][S44]

12.2 ADR 목록 (docs/adr/ 에 개별 파일)

ADR	결정	대안과 기각 이유	결과·부담
001	api(Java)와 collector(Python) 분리, Redis Streams로 연결	FastAPI 단일 프로세스(v0.1): 단순하지만 외부 지연이 사용자 경로와 같은 이벤트 루프를 공유, 실행 계획·부하 격리가 어려움. Java 단일: 어댑터 작성 속도가 Python보다 느림	언어 2개·계약 테스트 필요. 대신 장애 격리·확장 경로 (소비자 추가) 확보
002	Next.js 16 + BFF 프록시	Vite SPA: 가볍지만 CSP·프록시·키 은닉을 별도 서버로 만들어야 함	SSR 셀은 최소. 지도는 클라이언트 컴포넌트

ADR	결정	대안과 기각 이유	결과·부담
003	nginx edge 단일 공개 포트	Caddy: 자동 HTTPS가 장점이나 로컬에서는 불필요, 배포 시 재검토	XFF 신뢰 경계가 명확
004	PostGIS를 MVP부터	메모리+Redis만: 재생·통계·감사가 불가	Apple Silicon 이미지 우회 필요
005	외부 호출 예산을 Redis Lua로 중앙 관리	프로세스 내 카운터: 다중 작업·수동 호출에서 초과 위험	Redis 장애 시 예약 실패 → 수집 보수적 중단
006	api는 외부 API를 직접 호출하지 않음	요청 시 조회: 사용자 트래픽이 외부 한도를 소모	METAR는 watched 공항만 선제 수집
007	보존: 원해상도 72 h, 1분 요약 30일(관심 지역)	전체 30일: 로컬 디스크 초과	파티션·요약 배치 필요
008	WS 프로토콜은 원시 JSON + 버전 번호(STOMP 미사용)	STOMP: 구독 모델은 편하나 bbox별 diff·백프레시 제어가 어려움	프로토콜 문서·클라이언트 직접 구현
009	공개 배포에서는 OpenSky 기본 OFF	—(약관 [S3])	전세계 뷰는 로컬·동의 후에만
010	예측 알림은 관측 알림과 유형 분리, 항상 “추정” 표시	하나로 합치기: 정직성 위반	화면·API 필드 증가

13. 로컬 개발 환경 (macOS 26.6)

13.1 사전 확인

Homebrew 7.0.0 기준 Tahoe 26은 Apple Silicon에서 Tier 1이고 [S45], Intel Mac은 Tier 3(새 bottle 없음)이며 Tahoe는 Intel에서 도는 마지막 macOS입니다 [S46]. `uname -m` 으로 확인하세요.

13.2 설치

```
xcode-select --install
/bin/bash -c "$(curl -fsSL https://raw.githubusercontent.com/Homebrew/install/HEAD/install.sh)"
brew install git node@24 uv just k6
brew install --cask orbstack          # 개인 무료 [S43]; 또는 docker(Docker Desktop)
brew install --cask temurin@25        # JDK 25 (Adoptium 배포판; brew 캐스크 이름은 설치 시 확인)

git clone https://github.com/<you>/skywx && cd skywx
cp .env.example .env
just up                                # compose up --build (6 컨테이너)
just ops-user                          # 운영자 계정 생성(프롬프트)
open http://localhost:8080
```

불확실

JDK 25를 Homebrew로 설치하는 정확한 캐스크 이름(예: `temurin@25`)은 이 문서에서 확인하지 않았습니다. `brew search temurin` 으로 확인하거나 SDKMAN을 쓰세요.

13.3 저장소 구조

```
skywx/
├─ apps/
│  └─ api/                Spring Boot (Gradle Kotlin DSL)
│     └─ src/main/java/dev/skywx/{ingest,engine,ws,rest,persist,ops,config}
│  └─ collector/          Python (uv) skywx_collector/{providers,quality,publisher,budget,jobs}
│  └─ web/                Next.js 16 app/{(map),replay,stats,airports,ops,about}
├─ lib/{ws,store,interpolate.worker.ts}
├─ schemas/               aircraft_state.v1.json sigmet.v1.json stream_envelope.v1.json (계약의 단일 원천)
├─ infra/
│  └─ edge/nginx.conf db/init/roles.sql redis/redis.conf
│  └─ compose.yml compose.test.yml
├─ fixtures/              실응답 스냅샷(키 마스킹) — pytest·JUnit·E2E 공용
├─ docs/                  이 설계서, adr/ADR-001...010.md, VERIFICATION.md(검증 기록), PERF.md(측정 결과)
├─ .github/workflows/     ci.yml (lint·test·contract·e2e·security)
└─ justfile .env.example .pre-commit-config.yaml
```

13.4 compose.yml (핵심)

```
services:
  edge:
    image: nginx:1.27-alpine
    ports: ["8080:8080"]          # 유일한 공개 포트
    volumes: [".infra/edge/nginx.conf:/etc/nginx/nginx.conf:ro"]
    depends_on: [web, api]
  web:
```

```

build: ./apps/web
environment: { API_INTERNAL_URL: "http://api:8000" } # 서버 전용, NEXT_PUBLIC_ 없음
expose: ["3000"]
api:
  build: ./apps/api
  env_file: .env
  environment: { SPRING_THREADS_VIRTUAL_ENABLED: "true", MANAGEMENT_SERVER_PORT: "9000" }
  expose: ["8000", "9000"]
  depends_on: { redis: { condition: service_healthy }, db: { condition: service_healthy } }
  deploy: { resources: { limits: { memory: 1g } } }
  read_only: true
  tmpfs: ["/tmp"]
  cap_drop: ["ALL"]
collector:
  build: ./apps/collector
  env_file: .env # OPENSky_*, KMA_* 는 여기만
  volumes: ["raw:/data/raw"]
  depends_on: { redis: { condition: service_healthy }, db: { condition: service_healthy } }
  read_only: true
  cap_drop: ["ALL"]
redis:
  image: redis:8
  command: ["redis-server", "/etc/redis/redis.conf"] # requirepass, appendonly yes, maxmemory
  volumes: [".infra/redis/redis.conf:/etc/redis/redis.conf:ro", "redis_data:/data"]
  healthcheck: { test: ["CMD", "redis-cli", "-a", "${REDIS_PASSWORD}", "ping"], interval: 5s }
db:
  image: postgis/postgis:18-3.6
  platform: linux/amd64 # Apple Silicon: 공식 arm64 태그 없음 [S39][S40]
  environment: { POSTGRES_PASSWORD: "${DB_ROOT_PASSWORD}" }
  volumes: ["db_data:/var/lib/postgresql/data", ".infra/db/init:/docker-entrypoint-initdb.d:ro"]
  healthcheck: { test: ["CMD-USER", "pg_isready", "-U", "postgres"], interval: 5s }
volumes: { raw: {}, redis_data: {}, db_data: {} }

```

PostGIS 공식 이미지 18-3.6 은 2026-09-27 기준 amd64만 있고 [S39], 같은 문제를 겪은 프로젝트는 db 서비스에만 platform: linux/amd64 를 지정했습니다 [S40]. 에뮬레이션이 느리면 ARM64+AMD64 커뮤니티 이미지 [S41]나 postgres:18 + PostGIS 패키지 자체 빌드로 바꾸세요.

13.5 .env.example

DB_ROOT_PASSWORD, DB_API_PASSWORD, DB_COLLECTOR_PASSWORD	역할 3개 비밀번호(init/roles.sql이 생성)
REDIS_PASSWORD	requirepass
AIRCRAFT_PROVIDERS=adsb_lo!,adsb_fi,opensky	우선순위. 로컬은 OpenSky 포함 가능(연구·학습 범위), 배포는 ADR-009
OPENSky_CLIENT_ID / OPENSky_CLIENT_SECRET	OAuth2 클라이언트 [S1], collector에만 주입
REGION_CENTER=36.5,127.8 · REGION_RADIUS_NM=250 · REGION_POLL_S=10 · GLOBAL_POLL_S=120	런타임에 ops 설정으로 덮어씀
HTTP_USER_AGENT="skywx-dev/0.2 (contact: you@example.com)"	AWC 권고 [S12]
SKYWX_FIXTURE_MODE=0	1이면 외부 호출 없이 fixtures/ 재생

OPS_SESSION_TTL_H=8 · PUBLIC_RATE_LIMIT=120/min · WS_MAX_CONN=200	보안 파라미터
---	---------

13.6 justfile

```

up:      docker compose -f infra/compose.yml up --build -d
down:    docker compose -f infra/compose.yml down
logs svc: docker compose -f infra/compose.yml logs -f --tail=200 {{svc}}
test:    just test-api && just test-collector && just test-web
test-api: cd apps/api && ./gradlew test jacocoTestReport          # Testcontainers PostGIS·Redis
test-collector: cd apps/collector && uv run pytest -q --cov
test-web: cd apps/web && npm test
contract: uv run python tools/contract_check.py && cd apps/api && ./gradlew contractTest  # schemas/ 대조
e2e:     SKYWX_FIXTURE_MODE=1 just up && cd apps/web && npx playwright test
bench:   k6 run perf/rest.js && k6 run perf/ws.js
fixtures: uv run python -m skywx_collector.tools.snapshot        # 실응답 저장(키 마스킹), 한도 소모 주의
ops-user: docker compose exec api java -jar app.jar --create-ops-user
adr n:   cp docs/adr/template.md docs/adr/ADR-{{n}}.md

```

14. 테스트·품질 전략

14.1 테스트 피라미드

층	도구	대상(대표)	목표 수
collector 단위	pytest	정규화(단위:ground·null), 게이트 6규칙, 백오프·폴백 순서, 예산 Lua(가짜 Redis), SIGMET 파서 fixture 6종	≥ 60
collector 계약	pytest + respx + fixtures	어댑터별 실응답 → AircraftState 스키마 검증, 429/5xx 주입	≥ 15
api 단위	JUnit 5 · JMH	엔진 12케이스, 예측 ETA 오차, FSM 히스테리시스, diff, 카테고리 경계 8건, 마스킹 10패턴	≥ 80
api 통합	Testcontainers(PostGIS [S74] , Redis) · WireMock	스트림 소비→저장→REST, WS 구독·diff·재동기·백프레셔, Security(404/403/잠금), 파티션 잡, 역할 권한	≥ 40
계약(언어 간)	schemas/*.json	Python이 만든 메시지를 Java가 검증, Java가 만든 REST 응답을 Python 툴이 검증 — 한쪽만 고치면 CI 실패	≥ 6
web 단위	Vitest	보간·wrap180·stale, diff 적용, WS 재접속·resync 로직	≥ 25
E2E	Playwright(fixture 모드)	지도 로드·레이어·타임라인·알림 패널·근거 카드·attribution·지연 배지·ops 로그인·XFF 위조 3경로·CSP 위반 0	≥ 12
부하	k6	REST 100 rps, WS 200연결, 한도 초과 429, 느린 클라이언트 강제 재동기	3 시나리오
회귀 스모크	수동 체크리스트	실 API로 1시간: 예산 사용률, 429 0건, 게이트 격리율	—

14.2 CI 파이프라인 (GitHub Actions, PR·push)

lint(ruff·mypy·ESLint·spotless) → 단위 → 계약 → 통합(Testcontainers) → 웹 빌드 → E2E(fixture) → 보안 (gitleaks 전체 이력·CodeQL·Trivy·npm/pip audit) → 커버리지 게이트(80%). 공개 저장소 표준 러너는 무료 [\[S49\]](#).

14.3 검증 기록(VERIFICATION.md)

실데이터·부하·자동 검사로 찾은 문제를 “증상 → 재현 테스트 → 원인 → 수정 → 회귀 테스트” 순서로 번호를 붙여 기록한다. 기존 포트폴리오와 같은 형식이라 리뷰어가 읽기 쉽다.

15. 개발 로드맵 (WBS, 14주 · 주 10-15시간)

단계	주	작업 패키지	완료 기준
P0 준비	1	저장소·Compose 골격·역할 SQL·CI 린트, 커버리지 비교(3.6 불확실), schemas v1, ADR-001~004	just up 6컨테이너 healthy, 커버리지 메모
P1 수집	2-3	collector: 어댑터 3종·정규화·게이트·원천 보관·예산 Lua·XADD, ingest_run 기록	pytest 60+, fixture 15종, 예산 초과 테스트
P2 API 코어	4-5	api: 스트림 소비·스냅샷·REST(aircraft/sigmets/radar)·RFC 9457·요청 제한·ETag, Flyway V1, 항적 배치 저장·파티션 잡	Testcontainers 통합 20+, REST p95 예산
P3 실시간	6-7	WS 프로토콜·bbox 구독·diff·재동기·백프레셔, web 지도·보간 워커·레이더 타임라인·상태 바	NFR-01/03/04 측정, k6 WS 200
P4 판정	8-9	엔진(STRTree·FSM·예측), 알림 패널·근거 카드, SIGMET 레이어, alert_event	FR-08/09/10/21 테스트, JMH
P5 운영·보안	10-11	ops 로그인·화면·설정·감사, nginx XFF·제한, CSP, 마스킹, CodeQL·Trivy·gitleaks 게이트, 컨테이너 강화	NFR-10/11 통과, E2E 위조 헤더
P6 이력·통계·공항	12-13	재생 화면, 1분 요약, stats_daily, METAR/TAF·카테고리, 공항 레이어	FR-22/23/24, 재생 p95
P7 마무리	14	부하 시험 3회·PERF.md, VERIFICATION.md, README·데모 GIF, 포트폴리오 PDF, 배포용 설계서 v0.2 갱신	1.6 성공 기준 전부

P3까지 끝나면 “움직이는 항공기 + 레이더”가 보이는 데모가 나온다. 일정이 밀리면 P6의 통계·공항을 뒤로 미루고 P4·P5를 지킨다 (포트폴리오 핵심은 판정·운영·보안).

16. 리스크와 불확실 사항

리스크	가능성	대응
공급자 정책 변경(3.6절 이력)	높음	어댑터·폴백·fixture 모드·운영 화면 전환. 분기마다 약관 재확인
두 언어 유지 부담(ADR-001)	중간	스키마 파일이 단일 원천, 계약 테스트가 CI 게이트. 어댑터는 Python에만 추가
Apple Silicon에서 PostGIS 에뮬레이션 속도 [S39]	확실	커뮤니티 arm64 이미지 [S41] 또는 자체 빌드로 교체
한반도 커버리지 부족	중간	P0 비교 후 1순위 확정, 장기적으로 RTL-SDR 피딩
RainViewer 한국 레이더 미포함	중간	커버리지 마스크 확인 후 기상청 API허브(FR-31) 앞당김
예측 알림 오탐(선화·감속)	중간	트랙 변화 15° 초과 시 예측 보류, 오탐률을 통계 화면에 공개
디스크 10 GB 초과	낮음	보존 정책·파티션 삭제 잡, 운영 화면 디스크 경고
OpenSky 약관 해석(운영 목적) [S3]	중간	로컬은 연구·학습 범위, 공개 배포 시 OFF(ADR-009)
JDK 25·Spring Boot 4.1 생태계 호환 (라이브러리 지연)	낮음	문제 시 Java 21 LTS로 낮춤(코드 변경 없음)

이 문서에서 확인하지 못한 것

- 7.2절 용량 수치는 전부 추정입니다. P3 종료 시 실측으로 교체하세요.
- adsb.lol 응답에 등록번호(r)·기종(t)이 항상 포함되는지는 확인하지 못했습니다(readsb 필드 정의에는 있음 [S8]).
- OpenFreeMap 스타일이 참조하는 정확한 타일·글꼴 도메인은 배포 전 브라우저 콘솔로 확인해 CSP를 보정해야 합니다.

부록 A. 출처와 인용문

구분: 공식 = 서비스 운영자·개발자 문서·표준, 제3자 = 블로그·커뮤니티·타 프로젝트, 데이터 = 2026-09-27에 직접 조회한 API 응답. 인용문은 출처당 1개, 원문 그대로입니다. 본문의 그 밖의 수치·필드 목록은 같은 출처 페이지의 내용을 요약한 것입니다.

ID	구분	출처 / URL	인용문(원문, 2026-09-27 확인)
S1	공식	OpenSky Network REST API 문서 https://openskynetwork.github.io/opensky-api/rest.html	“OpenSky exclusively supports the OAuth2 client credentials flow.”
S2	공식	OpenSky Network API 문서(개요) https://openskynetwork.github.io/opensky-api/	“retrieve live airspace information for research and non-commercial purposes”
S3	공식	OpenSky Network 이용약관 https://opensky-network.org/about/terms-of-use	“requires a previous written agreement, even for non-profit or governmental entities”
S5	공식	OpenSky Python 클라이언트 README https://github.com/openskynetwork/opensky-api/tree/master/python	“basic authentication with username and password is no longer supported”
S6	공식	adsb.lol API 안내 https://www.adsb.lol/docs/open-data/api/	“The API is available to everyone.”
S7	공식	adsblol/api README https://github.com/adsblol/api	“Currently, there are no rate limits on the API.”
S8	공식	readsb README-json (aircraft.json 필드 정의) https://github.com/wiedehopf/readsb/blob/dev/README-json.md	“gs: ground speed in knots”
S9	공식	adsb.fi opendata README https://github.com/adsbf/opendata/blob/main/README.md	“The public endpoints are rate limited to 1 request per second”
S10	공식	airplanes.live API 가이드 https://airplanes.live/api-guide/	“Non-commercial access requires contacting Airplanes.Live”
S11	제3자	teddywagner/overhead PR #3 (2026-09) https://github.com/teddywagner/overhead/pull/3	“Airplanes.live now answers non-feeders with HTTP 403”
S12	공식	AviationWeather.gov Data API https://aviationweather.gov/data/api/	“Cross-origin resource sharing is not permitted at this time.”

ID	구분	출처 / URL	인용문(원문, 2026-09-27 확인)
S13	데이터	AviationWeather.gov isigmet JSON (2026-09-27 조회) https://aviationweather.gov/api/data/isigmet?format=json	“"hazard":"TS","qualifier":"EMBD","base":null,"top":48000,"geom":"AREA"”
S14	공식	Open-Meteo 이용약관 https://open-meteo.com/en/terms	“Less than 10'000 API calls per day, 5'000 per hour and 600 per minute.”
S15	공식	Open-Meteo 홈 https://open-meteo.com/	“No API key, no sign-up, no credit card.”
S16	공식	RainViewer API 전환 요약 https://www.rainviewer.com/api/transition-faq.html	“Maximum zoom level set to 7”
S17	공식	RainViewer Weather Maps API https://www.rainviewer.com/api/weather-maps-api.html	“Contains the past 2 hours of weather radar data with 10-minute intervals.”
S18	공식	RainViewer API 문서 https://www.rainviewer.com/api.html	“This API is available for personal and educational use only.”
S19	공식	기상청 API허브 이용안내 https://apihub.kma.go.kr/apiInfo.do	“회원가입을 통해 인증키를 발급받아야 합니다”
S20	공식	기상자료개방포털 레이어 합성 자료 https://data.kma.go.kr/data/rmt/rmtList.do?code=11&pgmNo=62&tabNo=2	“제공간격 10분→5분, 해상도 1km→500m”
S22	공식	MapLibre GL JS GitHub https://github.com/maplibre/maplibre-gl-js	“MapLibre GL JS is licensed under the 3-Clause BSD license.”
S23	공식	OpenFreeMap Quick Start https://openfreemap.org/quick_start/	“ https://tiles.openfreemap.org/styles/liberty ”
S24	공식	OpenFreeMap README https://github.com/hyperknot/openfreemap	“there are no limits on the number of map views or requests”
S25	공식	react-map-gl README https://github.com/visgl/react-map-gl	“npm install react-map-gl maplibre-gl”
S26	공식	카카오 데브톡 답변(2023-06) https://devtalk.kakao.com/t/map/129890	“카카오맵의 데이터 범위는 대한민국으로 한정됩니다.”
S27	제3자	InMotion Hosting, Node.js 26 릴리스(2026-05) https://www.inmotionhosting.com/support/news/nodejs-v26-released/	“Node.js 24 is the current Active LTS release”
S28	제3자	endoflife.date Node.js https://endoflife.date/nodejs	“Production applications should only use Active LTS or Maintenance LTS releases.”
S31	공식	Python 3.13.14 릴리스 https://www.python.org/downloads/release/python-31314/	“Python 3.13.14 is the fourteenth maintenance release of 3.13”
S33	공식	uv 문서 https://docs.astral.sh/uv/	“An extremely fast Python package and project manager, written in Rust.”
S35	공식	HTTPX https://www.python-httpx.org/	“Standard synchronous interface, but with async support if you need it.”

ID	구분	출처 / URL	인용문(원문, 2026-09-27 확인)
S37	제3자	Movable Type Scripts, 위경도 계산 https://www.movable-type.co.uk/scripts/latlong.html	“ δ is the angular distance d/R ”
S38	제3자	Wikipedia, Nautical mile https://en.wikipedia.org/wiki/Nautical_mile	“the international nautical mile is defined as exactly 1,852 metres”
S39	데이터	Docker Hub postgres/postgis 태그 API (2026-09-27 조회) https://hub.docker.com/v2/repositories/postgis/postgis/tags/?name=18-3.6	“18-3.6: amd64 + linux”
S40	제3자	AY2627S1-CS3219-P1/FoC 이슈 #13 https://github.com/AY2627S1-CS3219-P1/FoC/issues/13	“postgres/postgis:18-3.6 does not publish a linux/arm64 image.”
S41	제3자	lanechi/postgres-postgis https://github.com/lanechi/postgres-postgis	“Multi-architecture image (ARM64 + AMD64)”
S42	공식	Redis 라이선스 https://redis.io/legal/licenses/	“users can now choose the free and OSI-approved open-source AGPLv3 license”
S43	공식	OrbStack 요금 https://orbstack.dev/pricing	“Always free for personal use”
S44	공식	OrbStack 라이선스 문서 https://docs.orbstack.dev/licensing	“After the trial, it will degrade to Free (personal use only).”
S45	공식	Homebrew 7.0.0 발표(2026-09-13) https://brew.sh/2026/09/13/homebrew-7.0.0/	“Sequoia 15, Tahoe 26 and Golden Gate 27 are Tier 1 on Apple Silicon.”
S46	공식	Homebrew Support Tiers https://docs.brew.sh/Support-Tiers	“macOS Tahoe 26 is the final version of macOS to run on Intel hardware.”
S49	공식	GitHub Actions 요금 문서 https://docs.github.com/billing/managing-billing-for-github-actions/about-billing-for-github-actions	“GitHub Actions usage is free for self-hosted runners and for public repositories”
S53	공식	Flightradar24 API FAQ https://fr24api.flightradar24.com/docs/faq	“Currently, we do not offer any free-of-charge API access”
S54	공식	FlightAware AeroAPI 요금 https://flightaware.com/api	“up to \$5 free per month”
S55	공식	MapTiler Cloud 약관 https://www.maptiler.com/terms/cloud/	“limited to non-commercial use and research & development”
S56	제3자	Apify, Live Flight Tracker (OpenSky) 설명 https://apify.com/foo121/flight-tracker-opensky	“Coverage depends on OpenSky's volunteer receiver network”
S57	공식	OSMF 타일 이용정책 https://operations.osmfoundation.org/policies/tiles/	“Availability is best-effort: there is no SLA or guarantee.”
S58	공식	Docker Hub redis 공식 이미지 https://hub.docker.com/_/redis	“Starting with Redis 8.0, Redis follows a tri-licensing model”
S61	공식	react-map-gl 새 소식 https://visgl.github.io/react-map-gl/docs/whats-new	“The v8.1 release line supports MapLibre GL JS v4, v5 and v6.”

ID	구분	출처 / URL	인용문(원문, 2026-09-27 확인)
S62	공식	Redis Streams 문서 https://redis.io/docs/latest/develop/data-types/streams/	"A Redis stream is a data structure that acts like an append-only log"
S63	공식	Redis XREADGROUP 명령 문서 https://redis.io/docs/latest/commands/xreadgroup/	"a special version of the XREAD command with support for consumer groups"
S64	공식	Oracle, Java 25 출시 발표(2025-09-16) https://www.oracle.com/news/announcement/oracle-releases-java-25-2025-09-16/	"Oracle will provide long-term support for Java 25 for at least eight years"
S65	공식	Spring Boot 4.1.1 출시 공지(2026-08-20) https://spring.io/blog/2026/08/20/spring-boot-4-1-1-available-now/	"Spring Boot 4.1.1 has been released and is now available from Maven Central"
S66	공식	Spring Boot 문서, Task Execution(가상 스레드) https://docs.spring.io/spring-boot/reference/features/task-execution-and-scheduling.html	"using Java 21+ and spring.threads.virtual.enabled set to true"
S67	공식	Spring Framework WebSocket 서버 문서 https://docs.spring.io/spring-framework/reference/web/websocket/server.html	"the underlying standard WebSocket session (JSR-356) does not allow concurrent sending"
S68	공식	JTS STRtree Javadoc https://locationtech.github.io/jts/javadoc/org/locationtech/jts/index/strtree/STRtree.html	"A query-only R-tree created using the Sort-Tile-Recursive (STR) algorithm."
S69	공식	RFC 9457 Problem Details for HTTP APIs https://www.rfc-editor.org/rfc/rfc9457.html	"to avoid the need to define new error response formats for HTTP APIs"
S70	공식	AWC GFA Help(비행 카테고리 기준) https://aviationweather.gov/gfa/help/	"1000 feet to 3000 feet AGL and/or 3 to 5 miles"
S71	공식	PostGIS ST_Intersects 문서 https://postgis.net/docs/ST_Intersects.html	"Tests if two geometries intersect (they have at least one point in common)"
S72	공식	Next.js 16 업그레이드 가이드 https://nextjs.org/docs/app/guides/upgrading/version-16	"Minimum version now 20.9.0 (LTS); Node.js 18 no longer supported"
S73	공식	Grafana k6 WebSockets 문서 https://grafana.com/docs/k6/latest/using-k6/protocols/websockets/	"The k6/websockets module provides a more standard API than k6/ws."
S74	공식	Testcontainers PostGIS 모듈 https://testcontainers.com/modules/postgis/	"var postgis = new PostgreSQLContainer(image);"
S75	공식	OpenJDK JDK 25 프로젝트 페이지 https://openjdk.org/projects/jdk/25/	"2025/09/16 General Availability"

부록 B. 용어

ADS-B / ICAO24(hex)	항공기가 위치·속도를 방송하는 감시 방식 / 트랜스폰더 24비트 고유 주소(기본 키)
SIGMET · FIR · FL	비행에 중대한 기상 경보 · 비행정보구역(RKRR 인천) · 비행고도(FL350 = 35,000 ft)

METAR · TAF · 비행 카테고리	공항 정시 관측 · 공항 예보 · VFR/MVFR/IFR/LIFR(실링·시정 기준)
dead reckoning	마지막 위치·속도·방위로 현재/미래 위치를 추정
STRtree · PreparedGeometry	질의 전용 R-tree 공간 인덱스 · 반복 판정용 사전 처리 도형
Redis Streams · 소비자 그룹 · PEL	추가 전용 로그 · 그룹 단위 소비 · 확인 전 대기 목록(at-least-once)
BFF · edge · XFF	브라우저 대신 API를 부르는 화면 서버 · 유일한 공개 진입점 · X-Forwarded-For
RFC 9457	HTTP API 오류 응답 표준 형식(problem+json)
ADR	설계 결정 기록(배경·대안·결과)

부록 C. 자주 쓰는 명령

```
# OpenSky 토큰 [S1]
curl -s -X POST "https://auth.opensky-network.org/auth/realms/opensky-network/protocol/openid-connect/token" \
  -d grant_type=client_credentials -d client_id=$OPENSKY_CLIENT_ID -d client_secret=$OPENSKY_CLIENT_SECRET | jq -r
.access_token
# adsb.lol 한반도 250 NM / AWC 국제 SIGMET / RainViewer 프레임
curl -s "https://api.adsb.lol/v2/point/36.5/127.8/250" | jq '.ac | length'
curl -s -A "skywx-dev/0.2" "https://aviationweather.gov/api/data/isigmet?format=json" | jq length
curl -s https://api.rainviewer.com/public/weather-maps.json | jq '.radar.past | length'
# Redis 스트림 확인
docker compose exec redis redis-cli -a $REDIS_PASSWORD XINFO STREAM skywx:aircraft
docker compose exec redis redis-cli -a $REDIS_PASSWORD XPENDING skywx:aircraft api
# 파티션·용량
docker compose exec db psql -U postgres -c "SELECT relname, pg_size_pretty(pg_total_relation_size(oid)) FROM
pg_class WHERE relname LIKE 'track_point%' ORDER BY 1;"
```

외부 API 명령은 각 공급자 한도를 소모하므로 확인용으로 1~2회만 실행하세요.